

باسمه تعالی

عنوان پروژه

طراحی وبسایت فروشگاه پوشاک زنانه

خرداد ۱۴۰۵

تقدیم بہ

پدر بزرگوار و مادر مہربانم

کہ از نگاہشان صلابت

از رفتارشان محبت

و از صبرشان ایستادگی آموختم

پاسکزاری

پروردگار مایاری کن تا دانش اندکم ز ربانی باشد برای فزونی و تکبر و غرور، ز حلقه‌ای برای اسارت و ز دست‌آیی برای تجارت، بلکه گاهی باشد برای تحلیل از تو و تعالی ساختن زندگی خود و دیگران.

قبل از هر چیزی، خداوند بزرگ را به خاطر لطفی که بهواره شامل حال من نموده‌ام کرم. پس، از زحمات اساتید محترم راهنما، **جناب آقای دکتر** که نه تنها به عنوان استاد بلکه همچون بهکاری در تمام مراحل انجام این تحقیق از ر، بنمودها و کمک‌های بی‌دین ایشان بهره‌مند شده‌ام، بشکر و قدردانی می‌کنم.

چکیده

با توجه به رشد روزافزون تجارت الکترونیک و نیاز مشتریان به دسترسی آسان و سریع برای خرید محصولات مختلف نظیر پوشاک به صورت آنلاین، پروژه‌ی فروشگاه آنلاین یکپارچه با محوریت فروش پوشاک زنانه با نام “women-clothing-shop” طراحی و پیاده‌سازی شد. مسئله اصلی که این پروژه به آن پاسخ می‌دهد، رفع چالش‌های موجود در فرآیندهای فروش سنتی پوشاک از جمله مدیریت ناکارآمد موجودی، نبود ابزارهای تحلیلی مناسب برای مدیران و محدودیت دسترسی مشتریان به تجربه خرید آنلاین است. برای رسیدن به این هدف، از زبان برنامه‌نویسی پایتون و فریم‌ورک جنگو (نسخه‌ی ۶.x) به عنوان ستون فقرات توسعه استفاده شد تا پلتفرمی امن، مقیاس‌پذیر و قابل توسعه ایجاد شود.

در معماری این سامانه، چند اپلیکیشن مجزا توسعه یافته‌اند: core برای صفحات اصلی و تنظیمات پایه، accounts برای مدیریت کاربران و احراز هویت، shop برای مدل‌های محصولات و مدیریت کاتالوگ، و purchase برای سبد خرید و سفارش‌ها. ساختار داده‌ای دقیق این پروژه شامل موجودیت‌هایی مانند کاربر سفارشی (CustomUser) با سطوح دسترسی متفاوت، محصول (Clothe) با امکان تعریف انواع مختلف و واریانت‌ها (ClotheVariant) از نظر رنگ و سایز، سبد خرید مبتنی بر نشست (Cart) و سفارش (Order) و OrderItem است. فرآیندهای کلیدی مانند ثبت‌نام و ورود کاربر، ویرایش حساب کاربری، مدیریت محصولات و دسته‌بندی‌ها، سبد خرید هوشمند با پشتیبانی از کاربران مهمان و عضو، و فرآیند تسویه حساب امن به صورت کامل پیاده‌سازی شده‌اند.

برای بهبود تجربه کاربری، قالب‌های نمایشی HTML و CSS با ساختار واکنش‌گرا طراحی شده و فرم‌ها و نمای‌های مدیریت محصول و سفارش به مدیران امکان نظارت دقیق‌تر بر موجودی، قیمت‌گذاری و سفارش‌ها را می‌دهند. بخش مدیریت سفارشی این پروژه نیز امکان نمایش اطلاعات پایه سفارش‌ها، محصولات و وضعیت موجودی را فراهم کرده است.

نتیجه نهایی این پروژه، یک وب اپلیکیشن کاربردی و عملیاتی است که تجربه خریدی روان، شفاف و در دسترس را برای مشتریان فراهم می‌آورد و در عین حال، ابزاری قدرتمند برای مدیریت کسب‌وکار در اختیار ادمین قرار می‌دهد. این سامانه با رعایت اصول کدنویسی تمیز، مستندسازی مناسب و بهره‌گیری از راهکارهای امنیتی و حفظ یکپارچگی داده، به عنوان یک راهکار جامع برای دیجیتالی کردن عملیات فروش پوشاک زنانه عمل می‌کند.

فهرست مطالب

عنوان

صفحه

۱- فصل اول کلیات پروژه.....	۱
۱-۱- بیان کلیات مسئله.....	۱
۱-۲- نیازمندی‌های پروژه.....	۲
۱-۳- نرم افزارهای استفاده شده در پیاده سازی سیستم.....	۵
۱-۳-۱- فریم‌ورک جنگو (Django Framework).....	۵
۱-۳-۱-۲- زبان برنامه‌نویسی پایتون (Python).....	۶
۱-۳-۱-۳- پایگاه داده SQLite.....	۶
۱-۳-۱-۴- فناوری‌های فرانت‌اند (HTML, CSS, JavaScript).....	۷
۲- فصل دوم تحلیل و طراحی سیستم.....	۸
۲-۱- مقدمه.....	۸
۲-۲- نمودار مورد کاربرد (Use Case).....	۹
۲-۳- نمودار فعالیت (Activity).....	۱۱
۲-۴- نمودار توالی (Sequence Diagram).....	۱۳
۳- فصل سوم پیاده سازی سیستم.....	۱۵
۳-۱- مقدمه.....	۱۵
۳-۲- Python در پروژه.....	۱۶
۳-۲-۱- نصب Python.....	۱۶
۳-۲-۲- ساخت Virtual Environment و نصب کتابخانه‌ها.....	۱۷
۳-۲-۳- استفاده از IDE مناسب مانند VSCode.....	۱۷
۳-۲-۴- رعایت Clean Code.....	۱۸
۳-۲-۵- کامنت‌گذاری و مستندسازی.....	۱۹
۳-۳- Django در پروژه.....	۲۰
۳-۳-۱- راه‌اندازی یک پروژه جدید در Django.....	۲۰
۳-۳-۲- راه اندازی یک اپلیکیشن جدید در Django.....	۲۰
۳-۳-۳- انجام تنظیمات مناسب برای پروژه در Django.....	۲۱
۳-۳-۴- تعریف Model ها در Django.....	۲۱
۳-۳-۵- تعریف و کاربرد Forms.....	۲۷
۳-۳-۶- تعریف ساختار Templates.....	۳۲
۳-۳-۷- ارتباط Backend با Templates.....	۳۳

۳۳	۸-۳-۳- مسیرها (URLs) و مدیریت آن‌ها
۳۵	۹-۳-۳- تعریف Commands (دستورات سفارشی)
۳۵	۱۰-۳-۳- تعریف دایرکتوری Media
۳۶	۱۱-۳-۳- تعریف دایرکتوری Static
۳۶	۱۲-۳-۳- Django در Admin
۳۶	۳-۴- طراحی صفحات با HTML و CSS و Javascript
۳۸	۳-۵- پایگاه داده SQL در پروژه
۳۹	۱-۵-۳- ساختار پایگاه داده و ارتباط جداول
۴۳	۴- فصل چهارم نصب و راهنمای پروژه
۴۳	۴-۱- نحوه نصب و راه اندازی پروژه
۴۳	۱-۱-۴- ساخت محیط مجازی
۴۳	۲-۱-۴- نصب کتابخانه‌های مورد نیاز
۴۴	۳-۱-۴- انجام مایگریشن‌ها و ایجاد داده‌های پیش فرض
۴۴	۴-۱-۴- راه‌اندازی سرور توسعه
۴۴	۵-۱-۴- حذف فایل SQLite
۴۵	۶-۱-۴- استفاده از script.bat برای راحتی کار
۴۵	۴-۲- راهنمای سطوح دسترسی کاربران
۴۸	۴-۳- راهنمای صفحات پروژه
۴۸	۱-۳-۴- صفحات در دسترس عموم (بخش مهمان)
۵۳	۲-۳-۴- صفحات اختصاصی کاربر عضو
۵۵	۳-۳-۴- صفحات اختصاصی ادمین
۵۷	۴-۴- نتیجه گیری
۵۸	۴-۵- پیشنهادات
۵۹	۴-۶- منابع و مراجع

فهرست شکل ها

عنوان	صفحه
شکل ۱-۱: لوگوی Django	۶
شکل ۱-۲: لوگوی Python	۶
شکل ۱-۳: لوگوی SQLite	۷
شکل ۱-۴: لوگوی HTML, CSS و Javascript	۷
شکل ۲-۱: نمودار Use Case کاربر میهمان	۱۰
شکل ۲-۲: نمودار Use Case کاربر عضو سایت	۱۰
شکل ۲-۳: نمودار Use Case ادمین کاربر	۱۱
شکل ۲-۴: نمودار فعالیت ثبت نام کاربر	۱۲
شکل ۲-۵: نمودار توالی ورود کاربر به صفحه کاربری	۱۴
شکل ۳-۱: پایتون نصب شده	۱۶
شکل ۳-۲: برنامه‌ی VSCode	۱۸
شکل ۳-۳: نمونه کامنت گذاری ها	۱۹
شکل ۳-۴: پیاده سازی مدل CustomUser	۲۲
شکل ۳-۵: پیاده سازی مدل Clothe	۲۴
شکل ۳-۶: پیاده سازی مدل ClotheVariant	۲۴
شکل ۳-۷: پیاده سازی مدل Order	۲۶
شکل ۳-۸: پیاده سازی مدل OrderItem	۲۶
شکل ۳-۹: فرم ورود کاربران	۲۷
شکل ۳-۱۰: فرم ثبت نام کاربران (CustomCreationForm)	۲۸
شکل ۳-۱۱: فرم ویرایش اطلاعات کاربران (CustomUserEditForm)	۲۹
شکل ۳-۱۲: فرم ویرایش اطلاعات ادمین (AdminUserEditForm)	۳۰
شکل ۳-۱۳: ایجاد سفارش (OrderCreateForm)	۳۱
شکل ۳-۱۴: تمپلیت های استفاده شده در اپلیکیشن accounts	۳۳
شکل ۳-۱۵: فایل urls.py	۳۴
شکل ۳-۱۶: بخشی از فایل accounts/urls.py	۳۴
شکل ۳-۱۷: بخشی از پیاده سازی یک دستور سفارشی	۳۵
شکل ۴-۱: نمای script.bat	۴۵
شکل ۴-۲: نمای صفحه‌ی خانه	۴۹
شکل ۴-۳: نمای صفحه‌ی فروشگاه	۵۰

شکل ۴-۴ نمای صفحه‌ی جزئیات محصول.....	۵۱
شکل ۴-۵ نمای صفحه‌ی سبد خرید.....	۵۱
شکل ۴-۶ نمای صفحه‌ی ثبت‌نام.....	۵۲
شکل ۴-۷ نمای صفحه‌ی داشبورد.....	۵۳
شکل ۴-۸ نمای صفحه‌ی پرداخت.....	۵۴
شکل ۴-۹ نمای صفحه‌ی تاریخچه سفارشات.....	۵۵
شکل ۴-۱۰ نمایی از داشبورد ادمین.....	۵۶
شکل ۴-۱۱: نمایی از صفحه ویرایش ادمین.....	۵۶

فصل اول

کلیات پروژه

۱-۱- بیان کلیات مسئله

در دنیای امروز، گذار کسب و کارها به پلتفرم‌های دیجیتال یک ضرورت است و حوزه خرده‌فروشی پوشاک از این قاعده مستثنی نیست. فروشگاه‌های سنتی با چالش‌هایی همچون دسترسی محدود مشتریان، فرایندهای دستی مدیریت موجودی و سفارشات، و فقدان ابزارهای تحلیلی مواجه‌اند که هم کیفیت سرویس‌رسانی و هم تصمیم‌گیری تجاری را تضعیف می‌کند. مشتریان نیز انتظار تجربه‌ای سریع، قابل اعتماد و شفاف هنگام جستجو، انتخاب و خرید پوشاک دارند به ویژه وقتی که محصول شامل انواع مختلفی بر حسب متغیرهایی مانند رنگ و سایز می‌شود.

پروژه «women-clothing-shop» به منظور پاسخ به این نیازها طراحی و پیاده‌سازی شده است. مسئله اصلی، ایجاد یک سامانه یکپارچه فروش آنلاین پوشاک زنانه است که فرایندهای کلیدی تجارت خرده‌فروشی را خودکار و قابل پیگیری کند، دسترسی کاربران را تسهیل نماید و ابزارهای مدیریتی و تحلیلی قدرتمندی در اختیار ادمین قرار دهد. بخش‌های اصلی مسئله عبارت‌اند از:

الف) چالش‌های سمت کاربران:

- محدودیت دسترسی و اطلاعات: نیاز به مشاهده جزئیات محصول (تصاویر، توضیحات، اندازه‌ها، رنگ‌ها و مشخصات) در هر زمان و مکان.
- پیچیدگی در انتخاب و تطبیق واریانت‌ها: کاربر باید بتواند به‌سادگی رنگ و سایز مناسب را انتخاب کند و از موجودی آن ترکیب مطلع شود.

- نبود امکان پیگیری سفارش: کاربران باید بتوانند وضعیت سفارش‌های خود را (مثلاً «در حال پردازش»، «ارسال شده»، «تحویل شده») پیگیری کنند و تاریخچه سفارشات را مشاهده نمایند.

ب) چالش‌های سمت کسب‌وکار (مدیریت فروشگاه):

- مدیریت موجودی و جلوگیری از فروش‌های بیش از حد: نیاز به به‌روزرسانی خودکار موجودی پس از هر سفارش موفق و اطلاع‌رسانی درباره محصولات با موجودی پایین.
- فقدان داده‌ها و ابزار تحلیلی: ضرورت نمایش گزارش‌های فروش، روندهای روزانه و محصولات پرفروش برای تصمیم‌گیری مبتنی بر داده.
- تفکیک منطقی محصول/واریانت: مدیریت جداگانه‌ی محصول اصلی و واریانت‌هایش (رنگ/سایز/موجودی) به‌منظور کنترل دقیق‌تر انبار و قیمت‌گذاری.

در نتیجه، هدف پروژه طراحی سیستمی است که با خودکارسازی فرایندها، ارائه اطلاعات شفاف به مشتریان و فراهم‌آوری داشبورد مدیریتی تحلیلی، فاصله میان فروش سنتی و نیازمندی‌های دیجیتال را پر کند.

۲-۱- نیازمندی‌های پروژه

نیازمندی‌ها به دو گروه کارکردی و غیرکارکردی تقسیم می‌شوند. نیازمندی‌های کارکردی، عملیات و قابلیت‌هایی که سیستم باید ارائه دهد را توصیف می‌کنند، در حالی که نیازمندی‌های غیرکارکردی، کیفیت و ویژگی‌های عملکردی سیستم را مشخص می‌سازند.

الف) نیازمندی‌های کارکردی (Functional Requirements)

۱. مدیریت کاربران و احراز هویت

- سیستم باید ثبت‌نام کاربران جدید با نام کاربری و ایمیل یکتا را پشتیبانی کند.
- امکان ورود با نام کاربری یا ایمیل باید در دسترس باشد.
- دو سطح دسترسی اصلی: «مشتری» و «ادمین» باید تعریف شود.
- کاربران واردشده باید به داشبورد شخصی دسترسی داشته باشند تا پروفایل (نام، نام‌خانوادگی، تلفن) را مدیریت و رمز عبور را تغییر دهند.
- خروج امن (logout) باید فراهم باشد.

۲. مدیریت محصولات و دسته‌بندی‌ها

- محصولات باید دارای نام، دسته‌بندی، رنگ، سایز، توضیحات، قیمت، درصد تخفیف، موجودی و تصاویر باشند.
- امکان تعریف دسته‌بندی برای سازمان‌دهی محصولات باید وجود داشته باشد.
- محصول باید بتواند دارای واریانت (رنگ/سایز) باشد؛ هر واریانت دارای موجودی مستقل است.
- نمایش لیست محصولات با قابلیت فیلتر (دسته‌بندی) و مرتب‌سازی (نام، قیمت، جدیدترین) باید فراهم شود.
- هر محصول باید صفحه جزئیات جداگانه با تصاویر گالری، واریانت‌ها و اطلاعات اندازه‌ها داشته باشد.
- قابلیت جستجوی متنی روی عنوان و توضیحات محصولات باید پیاده‌سازی شود.

۳. مدیریت سبد خرید

- پشتیبانی از سبد خرید مبتنی بر نشست (session) برای کاربران مهمان.
- برای کاربران عضو، سبد خرید مرتبط با حساب کاربری باقی بماند.
- هنگام ورود کاربر، سبد موقت مهمان باید با سبد حساب او ادغام شود.
- عملیات افزودن، ویرایش تعداد، حذف آیتم و پاک‌کردن سبد باید ممکن باشد.
- جلوگیری از افزودن بیش از موجودی برای هر واریانت باید تضمین شود.
- نمایش شمارش کل اقلام سبد در هدر یا نوار ناوبری در تمام صفحات.

۴. فرآیند سفارش و پرداخت

- تنها کاربران احراز هویت‌شده مجاز به دسترسی به صفحه نهایی کردن خرید باشند.
- هنگام ثبت سفارش، قیمت هر آیتم باید در لحظه خرید ثبت شود تا تغییرات بعدی قیمت بر سفارش اثر نگذارد.
- پس از ثبت سفارش موفق، رکورد سفارش با شماره پیگیری یکتا تولید شود.
- موجودی واریانت‌ها پس از هر سفارش موفق به‌صورت خودکار کاهش یابد.
- کاربران باید قابلیت مشاهده تاریخچه و جزئیات سفارش‌های قبلی را داشته باشند.

۵. پنل مدیریت (Admin)

- داشبورد مدیریت سفارشی با گزارش‌های کلیدی: مجموع فروش، تعداد سفارشات، سفارشات در حال پردازش، محصولات پرفروش و فروش روزانه.
- مدیریت کامل مدل‌ها: کاربران، محصولات، واریانت‌ها، سفارشات (فقط توانایی تغییر وضعیت سفارش) و تصاویر.
- امکان تغییر وضعیت سفارش‌ها (مثلاً به «ارسال شده») توسط ادمین.
- ابزارهای جستجو و فیلتر در بخش محصولات برای کارآمدی کار مدیریتی.

ب) نیازمندی‌های غیرکارکردی (Non-functional Requirements)

۱. امنیت (Security)

- رمزهای عبور باید با الگوریتم‌های امن هش شده در پایگاه داده نگهداری شوند.
- صفحات حساس (داشبورد، تکمیل خرید، مدیریت سفارش) باید محدود به کاربران مجاز باشند.
- محافظت در برابر XSS، CSRF و حملات رایج وب باید فعال و بررسی شود.
- دسترسی کاربران به اطلاعات سفارش دیگران باید ممنوع باشد.

۲. یکپارچگی داده (Data Integrity)

- عملیات تراکنشی حساس (ایجاد سفارش، کاهش موجودی، پاک‌سازی سبد) باید درون تراکنش‌های اتمیک انجام شود تا از ناسازگاری جلوگیری شود.
- حذف یک محصول که در سفارشات ثبت شده وجود دارد باید مسدود یا منطق نگهداری تاریخیچه را رعایت کند.
- هر محصول (واریانت) فقط یک رکورد در سبد یک کاربر داشته باشد؛ افزودن مجدد باید باعث افزایش تعداد شود نه ایجاد آیتم تکراری.

۳. قابلیت نگهداری و توسعه (Maintainability & Extensibility)

- کد باید مستندسازی شده و Docstring برای کلاس‌ها و توابع اصلی وجود داشته باشد.
- ساختار ماژولار پروژه (apps: core, accounts, shop, purchase) باید امکان توسعه آینده را ساده کند.

- اسکرپت‌های تولید داده نمونه برای راه‌اندازی محیط توسعه و تست (seeders) فراهم باشد.
- وابستگی‌ها در requirements.txt یا معادل مدیریت شوند و راهنمای راه‌اندازی در مستندات پروژه موجود باشد.

۴. قابلیت استفاده (Usability)

- آدرس‌ها (URLs) خوانا و قابل فهم برای کاربر باشند.
 - رابط کاربری واکنش‌گرا باشد و روی موبایل تجربه مناسبی ارائه دهد.
 - فرم‌ها و پیام‌های خطا به صورت واضح و کاربرپسند نمایش داده شوند (مثلاً اطلاع‌رسانی در مورد موجودی ناکافی).
 - فرایند خرید باید تا حد امکان با حداقل کلیک و به صورت مستقیم و شفاف انجام شود.
- این مجموعه نیازمندی‌ها مسیر طراحی و پیاده‌سازی "women-clothing-shop" را مشخص می‌سازد و مبنای تست و ارزیابی کارکردهای سیستم در فازهای بعدی توسعه خواهد بود.

۱-۳- نرم افزارهای استفاده شده در پیاده سازی سیستم

برای توسعه و پیاده‌سازی پروژه فروشگاه آنلاین پوشاک زنانه "women-clothing-shop"، از مجموعه‌ای از زبان‌های برنامه‌نویسی، فریم‌ورک‌ها، کتابخانه‌ها و ابزارهای نرم‌افزاری استفاده شده است. هر یک از این ابزارها نقش کلیدی در یکی از لایه‌های سیستم، از پایگاه داده و منطق سمت سرور گرفته تا رابط کاربری، ایفا می‌کنند. در ادامه، ابزارهای اصلی به کار رفته به تفکیک معرفی می‌شوند.

۱-۳-۱- فریم‌ورک جنگو (Django Framework)

جنگو، یک فریم‌ورک وب سطح بالای پایتون، به عنوان شالوده اصلی این پروژه عمل می‌کند. معماری MVT (Model-View-Template) جنگو برای سازماندهی ساختار پروژه استفاده شده است. قابلیت‌های کلیدی جنگو مانند ORM قدرتمند برای تعامل با پایگاه داده، سیستم مسیریابی URL، پنل مدیریت خودکار و سیستم پردازش فرم‌ها، سرعت و کیفیت توسعه را به شکل چشمگیری افزایش داده‌اند. پنل مدیریت این پروژه نیز به طور کامل با استفاده از امکانات جنگو سفارشی‌سازی شده است.



شکل ۱-۱ لوگوی Django

۱-۳-۲- زبان برنامه‌نویسی پایتون (Python)

پایتون به عنوان زبان اصلی و هسته مرکزی توسعه این پروژه به کار رفته است. تمام منطق سمت سرور، پردازش داده‌ها، تعامل با پایگاه داده و قوانین کسب‌وکار با استفاده از این زبان پیاده‌سازی شده است. انتخاب پایتون به دلیل سادگی، خوانایی بالا، و اکوسیستم قدرتمند آن، به ویژه در زمینه توسعه وب با فریم‌ورک جنگو، صورت گرفته است. این زبان به توسعه‌دهنده اجازه می‌دهد تا با کدنویسی کمتر، قابلیت‌های پیچیده‌ای را به شکلی تمیز و قابل نگهداری پیاده‌سازی کند.



شکل ۱-۲ لوگوی Python

۱-۳-۳- پایگاه داده SQLite

SQLite به عنوان سیستم مدیریت پایگاه داده پیش‌فرض در فاز توسعه این پروژه انتخاب شده است. این پایگاه داده به دلیل سادگی، عدم نیاز به سرور مجزا و پیکربندی آسان، گزینه‌ای ایده‌آل برای راه‌اندازی سریع محیط توسعه و تست است. تمام مدل‌های داده‌ای پروژه، از جمله کاربران، محصولات و سفارشات، از طریق ORM جنگو بر روی این پایگاه داده ذخیره و مدیریت می‌شوند. ساختار پروژه به گونه‌ای است که امکان مهاجرت به پایگاه داده‌های قدرتمندتر مانند PostgreSQL در فاز استقرار نهایی به راحتی وجود دارد.



شکل ۳-۱ لوگوی SQLite

۱-۳-۴ - فناوری‌های فرانت‌اند (HTML, CSS, JavaScript)

این سه فناوری، هسته اصلی لایه نمایش و رابط کاربری وبسایت را تشکیل می‌دهند. کدهای HTML برای ساختاردهی صفحات، CSS برای استایل‌دهی و زیباسازی، و JavaScript برای ایجاد تعاملات پویای سمت کاربر (مانند نمایش نام فایل عکس انتخابی) به کار رفته‌اند. جنگو از طریق سیستم Template خود، کدهای HTML را به صورت پویا رندر می‌کند و کلاس‌های CSS در فرم‌ها برای استایل‌دهی ویجت‌ها استفاده شده‌اند.



شکل ۴-۱ لوگوی HTML، CSS و Javascript

فصل دوم

تحلیل و طراحی سیستم

۲-۱- مقدمه

فصل تحلیل و طراحی به عنوان یکی از مراحل کلیدی در توسعه نرم افزار، نقشی اساسی در تضمین موفقیت پروژه فروشگاه آنلاین پوشاک زنانه "women-clothing-shop" ایفا می کند. در این فصل، با استفاده از تکنیک ها و ابزارهای استاندارد مدل سازی، به شناسایی نیازمندی ها و طراحی ساختار سیستم پرداخته می شود. هدف اصلی این فصل، تجزیه و تحلیل دقیق الزامات عملکردی و غیرعملکردی سیستم و طراحی مدل های مفهومی است که به تیم توسعه کمک می کند تا یک دیدگاه روشن و مشترک از معماری، عملکرد و ویژگی های سیستم مورد نظر داشته باشند.

در این فصل، ابتدا به بررسی نمودار مورد کاربرد (Use Case) خواهیم پرداخت که نمای کلی از تعاملات بین بازیگران مختلف سیستم (مهمان، مشتری و ادمین) و قابلیت های ارائه شده توسط پلتفرم "women-clothing-shop" را نمایش می دهد. سپس، نمودار فعالیت (Activity) به ما کمک می کند تا جریان کار و فرآیندهای اصلی سیستم، مانند ثبت نام کاربر و ثبت سفارش، را با جزئیات کامل شناسایی کنیم. در ادامه، نمودار توالی (Sequence) به توصیف نحوه تعامل اجزای مختلف سیستم در طول زمان می پردازد. این نمودارها، به عنوان ابزارهای بصری و تحلیلی، ما را در درک بهتر نیازمندی ها و طراحی یک سیستم کارآمد و مقیاس پذیر یاری می کنند.

۲-۲- نمودار مورد کاربرد (Use Case)

این نمودار تعاملات میان بازیگران (Actors) و سیستم را برای دستیابی به اهداف مشخص نمایش می‌دهد. این نمودار دید سطح بالایی از عملکرد سیستم ارائه داده و به شناخت نیازمندی‌های کاربران کمک می‌کند. برای پروژه "women-clothing-shop"، سه بازیگر اصلی با موارد استفاده مشخص تعریف شده‌اند:

۱- بازیگر (Actor)

به فرد یا موجودیتی خارجی که با سیستم ما در تعامل است، بازیگر گفته می‌شود.

۲- مورد کاربرد

نمایش‌دهنده عملیاتی است که توسط بازیگر در سیستم انجام می‌شود و با یک هدف مشخص پایان می‌یابد.

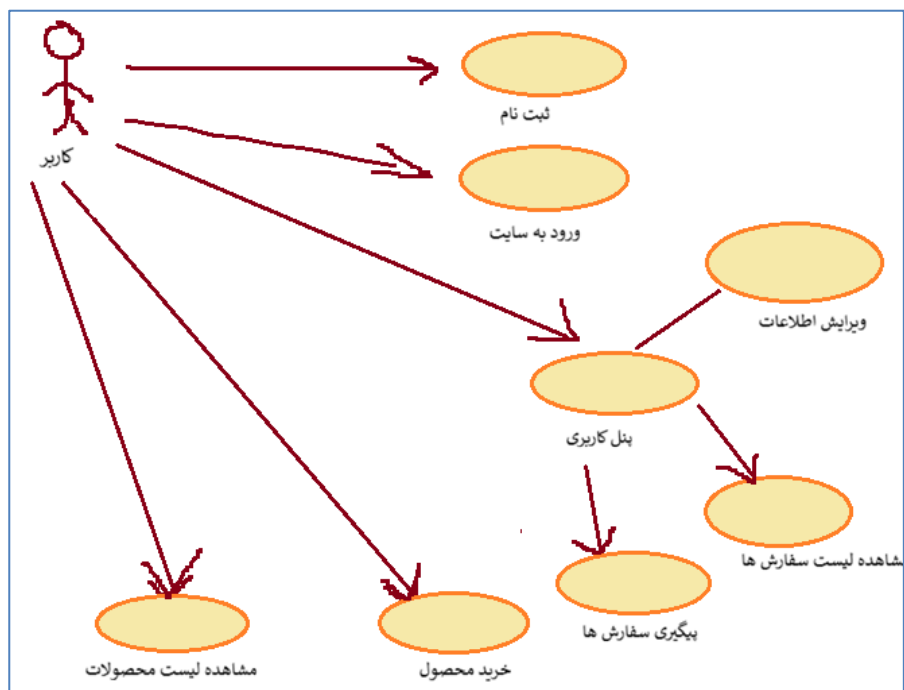
۳- موارد کاربرد پروژه women-clothing-shop:

- کاربر مهمان (Guest): فردی که بدون ورود به حساب کاربری از سایت بازدید می‌کند.
 - مشاهده صفحات اصلی: شامل صفحه خانه و درباره ما.
 - مرور محصولات: مشاهده لیست کامل محصولات، فیلتر بر اساس قیمت و مرتب‌سازی.
 - جستجوی محصول: جستجو در میان محصولات بر اساس نام و دسته‌بندی لباس.
 - مرتب‌سازی لیست محصولات: مرتب‌سازی محصولات بر اساس جدیدترین، ارزانترین و یا گرانترین.
 - مشاهده جزئیات محصول: دیدن اطلاعات کامل یک لباس خاص و افزودن یک ترکیب خاص از لباس (شامل سایز و رنگ مشخص) به سبد خرید.
 - مدیریت سبد خرید موقت: افزودن، ویرایش و حذف محصول از سبد خرید مبنی بر نشست (Session). امکان تکمیل فرایند خرید تا قبل از ورود وجود ندارد.
 - ثبت‌نام در سایت: ایجاد حساب کاربری جدید.
 - ورود به سایت: ورود به حساب کاربری موجود.



شکل ۱-۲: نمودار Use Case کاربر میهمان

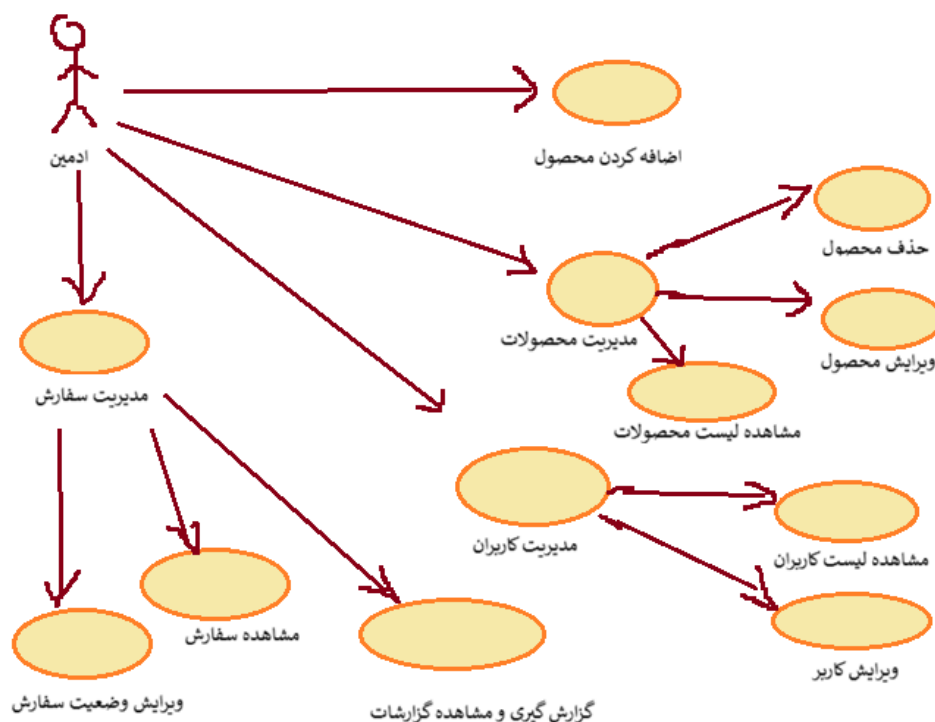
- کاربر عضو (Registered Customer): کاربری که با حساب خود وارد سیستم شده است. این بازیگر علاوه بر تمام موارد استفاده کاربر مهمان، به قابلیت‌های زیر دسترسی دارد:
 - مدیریت پروفایل: به‌روزرسانی اطلاعات شخصی، تغییر رمز عبور و تغییر شماره تلفن.
 - ثبت سفارش: نهایی کردن خرید.
 - مشاهده تاریخچه سفارشات: دسترسی به لیست کامل سفارش‌های قبلی و وضعیت آن‌ها.
 - خروج از سیستم: خروج امن از حساب کاربری.



شکل ۲-۲: نمودار Use Case کاربر عضو سایت

- کاربر ادمین (Admin): کاربری با بالاترین سطح دسترسی که مسئولیت مدیریت کل سیستم را بر عهده دارد.

- دسترسی به داشبورد مدیریت: ورود به پنل مدیریت.
- گزارش گیری: مشاهده گزارش های کلیدی مانند تعداد فروش، مجموع درآمد، محصولات پرفروش و فروش روزانه.
- مدیریت محصولات: افزودن، ویرایش و حذف محصولات.
- مدیریت سفارشات: مشاهده جزئیات سفارش ها و تغییر وضعیت آن ها (مثلاً به "ارسال شده").
- مدیریت کاربران: مشاهده لیست کاربران و مدیریت اطلاعات آن ها.

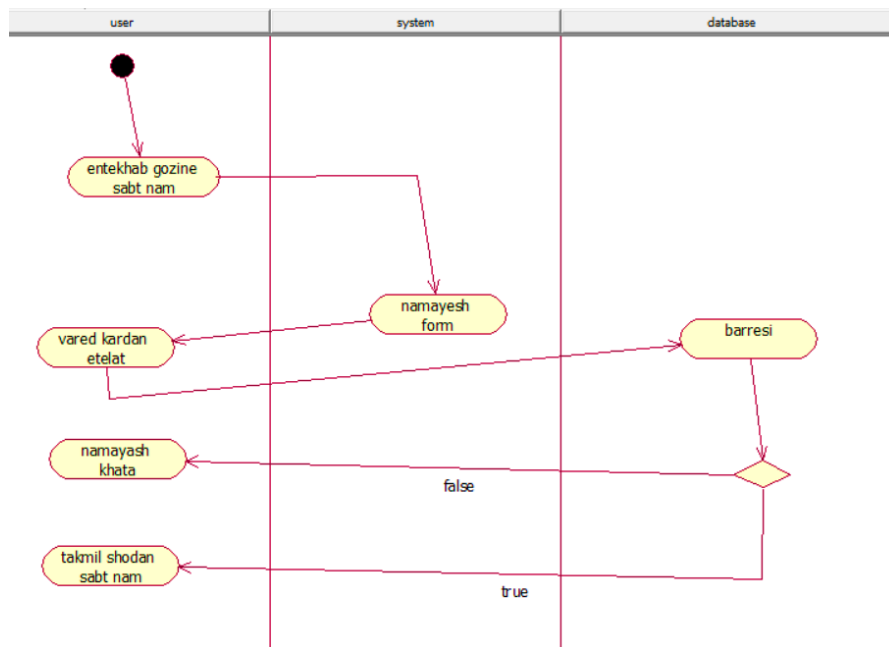


شکل ۳-۲: نمودار Use Case ادمین کاربر

۳-۲- نمودار فعالیت (Activity)

- در این نمودار، جریان گام به گام فرآیندهای اصلی سیستم به تصویر کشیده می شود. این نمودار بر روی توالی فعالیت ها، نقاط تصمیم گیری و جریان کنترل تمرکز دارد.
- **نمودار فعالیت ثبت نام کاربر:** این فرآیند از زمانی آغاز می شود که کاربر فرم ثبت نام را پر می کند. سیستم ابتدا معتبر بودن داده ها را بررسی می کند؛ از جمله اینکه آیا نام کاربری تکراری هست یا خیر.

در صورت وجود خطا، پیام مناسب به کاربر نمایش داده شده و فرآیند در همان مرحله باقی می ماند. اگر داده ها معتبر باشند، یک نمونه جدید از کلاس CustomUser در پایگاه داده ایجاد شده، رمز عبور هاش می شود و کاربر به صفحه ورود هدایت می گردد.



شکل ۴-۲: نمودار فعالیت ثبت نام کاربر

• نمودار فعالیت ثبت سفارش:

۱. پس از کلیک کاربر وارد شده روی دکمه «نهایی کردن خرید»، فرایند ثبت سفارش آغاز می‌شود.
۲. کلیه عملیات ثبت سفارش داخل یک تراکنش پایگاه داده (atomic transaction) اجرا می‌شود تا از یکپارچگی داده‌ها و همزمانی صحیح جلوگیری شود.
۳. ابتدا موجودی انبار برای تمام واریانت‌های کالا که در سبد خرید وجود دارد بررسی می‌شود. این با قفل کردن (select_for_update) مقادیر ClotheVariant انجام می‌شود تا از فروش بیش‌ازحد جلوگیری گردد.
۴. اگر همه آیتم‌ها موجود باشند، یک رکورد جدید Order ایجاد می‌شود و اطلاعات حمل‌ونقل/آدرس کاربر در آن ذخیره می‌گردد.
۵. سپس برای هر آیتم در سبد، یک OrderItem ساخته می‌شود و قیمت نهایی در زمان خرید در آن ثبت می‌شود تا تغییرات بعدی قیمت روی سفارش‌های گذشته اثر نگذارد.
۶. موجودی هر واریانت مطابق با تعداد سفارش شده کاهش می‌یابد.
۷. در پایان سبد خرید کاربر پاک می‌شود و کاربر به صفحه تأیید سفارش یا تاریخچه سفارش‌ها هدایت می‌گردد.

این فرایند تضمین می‌کند که:

- هیچ سفارشی بدون بررسی موجودی ثبت نشود.
- مقدار موجودی همیشه همگام با سفارش‌ها باقی بماند.
- عملیات ثبت سفارش اتمیک باشد و در صورت بروز خطا، تغییرات نیمه‌تمام برگشت داده شوند.

۲-۴- نمودار توالی (Sequence Diagram)

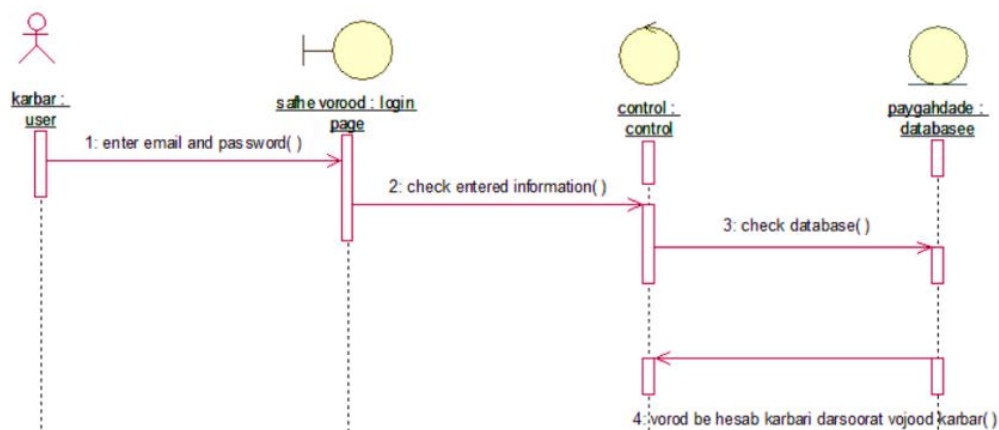
این نمودار تعاملات بین اشیاء و اجزای مختلف سیستم را در طول زمان نشان می‌دهد و بر ترتیب ارسال و دریافت پیام‌ها (فراخوانی متدها) تمرکز دارد.

اجزای نمودار توالی عبارتند از:

۱. خطوط حیات (Lifelines): هر شیء شرکت‌کننده در نمودار توالی، با یک خط عمودی نقطه چین و یک مستطیل در بالای آن نمایش داده می‌شود. این خط نشان دهنده طول عمر شیء در طول تعامل است.

۲. پیام‌ها (Messages): پیام‌ها نمایش دهنده‌ی تعاملات بین اشیاء هستند. یک پیکان از یک خط حیات به خط حیات دیگر نشان‌دهنده‌ی فرستادن یک پیام (معمولاً فراخوانی یک متد) از یک شیء به شیء دیگر است.

نمودارهای توالی ارائه شده، تعامل کاربر (مدیر) با سیستم را نشان می‌دهند. در نمودار توالی مدیریت کاربر (مدیر) پس از ورود به سیستم و انتخاب گزینه‌ی افزودن محصول، می‌تواند اقدام به افزودن کالا نماید و هنگامی که ادمین صفحه مربوط به یک محصول را باز کند، بجای دیدن صفحه محصول (صفحه‌ای که کاربر عادی می‌بیند) با فرم ویرایش کالا مواجه می‌شود که می‌تواند اقدام به ویرایش اطلاعات و یا حذف محصول نماید.



شکل ۵-۲: نمودار توالی ورود کاربر به صفحه کاربری

فصل سوم

پیاده سازی سیستم

۳-۱- مقدمه

در این فصل، به تشریح فرآیند و جزئیات فنی پیاده سازی پروژه فروشگاه آنلاین پوشاک زنانه **women-clothing-shop** پرداخته می شود. این سیستم با تکیه بر مجموعه ای از فناوری های مدرن و کارآمد توسعه یافته است تا نیازمندی های تحلیل شده در فصل پیشین را به طور کامل پوشش دهد. انتخاب ابزارها بر مبنای کارایی، امنیت، و قابلیت توسعه پذیری صورت گرفته است.

زبان برنامه نویسی پایتون (Python) به عنوان زبان اصلی برای توسعه تمام منطق سمت سرور، از تعریف مدل های داده ای در فایل های `models.py` گرفته تا پیاده سازی منطق پیچیده کسب و کار در نماها (Views)، به کار رفته است. بر روی این بستر، فریم ورک جنگو (Django) نسخه ۶.X به عنوان ستون فقرات اصلی پروژه، ساختاری منظم و قدرتمند را فراهم کرده است. از قابلیت های کلیدی جنگو در این پروژه می توان به موارد زیر اشاره کرد:

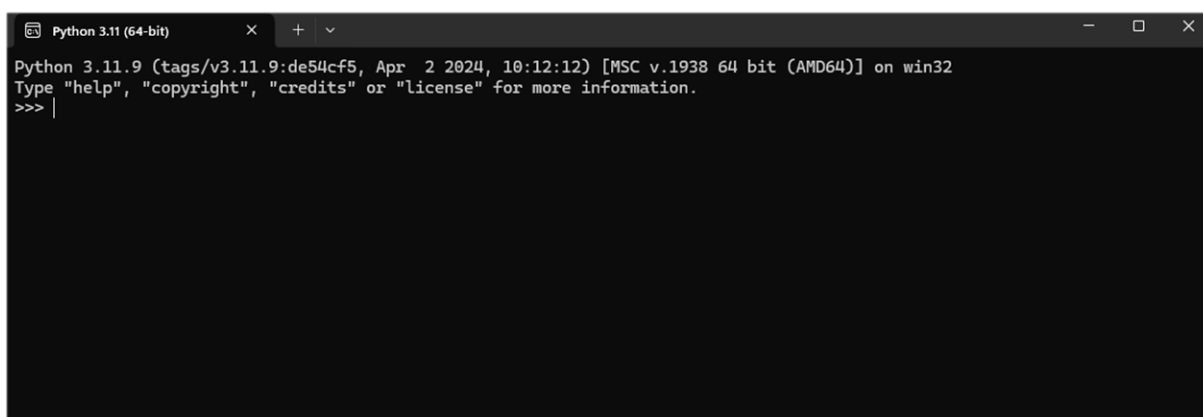
- **ORM جنگو** برای مدل سازی موجودیت هایی مانند `CustomUser`، `Clothe` و `Order` و مدیریت تمام تعاملات با پایگاه داده.
- **سیستم Template** برای رندر کردن پویای صفحات HTML و نمایش داده ها به کاربر.
- **مکانیزم Form Handling** برای اعتبارسنجی ورودی های کاربر در بخش هایی نظیر ثبت نام و ورود.
- **پنل مدیریت (Admin)** که برای مدیریت کاربران و محصولات توسط ادمین می باشد.

در لایه پایگاه داده، از **SQLite** به عنوان یک راهکار سبک و کارآمد برای محیط توسعه استفاده شده است. ولی پیاده‌سازی‌ها به صورتی هستند که در آینده به راحتی بتوان به پایگاه داده‌های حرفه‌ای تر مانند **PostgreSQL** نیز مهاجرت کرد. لایه نمایش یا فرانت‌اند سیستم نیز با استفاده از فناوری‌های استاندارد وب شامل **HTML** برای ساختاردهی، **CSS** برای استایل‌دهی (که از طریق کلاس‌ها در فرم‌های جنگو اعمال شده) و **JavaScript** برای بهبود تعاملات کاربر پیاده‌سازی شده است. ترکیب این ابزارها در کنار یکدیگر، منجر به خلق یک سیستم نرم‌افزاری یکپارچه، امن و قابل نگهداری شده است که به طور مؤثر به نیازهای کاربران و مدیران سیستم پاسخ می‌دهد.

۲-۳- Python در پروژه

۳-۲-۱- نصب Python

برای نصب Python، ابتدا باید به وب‌سایت رسمی Python مراجعه کرده و نسخه مناسب سیستم‌عامل خود را دانلود کنید. پس از دانلود، فایل نصب را اجرا کرده و مراحل نصب را دنبال کنید. در این مرحله، اطمینان حاصل کنید که گزینه "Add Python to PATH" را فعال کنید تا بتوانید به راحتی از Python در خط فرمان و یا ترمینال خود استفاده کنید. پس از اتمام نصب، با باز کردن ترمینال یا Command Prompt (CMD) و وارد کردن دستور `python --version` می‌توانید از موفقیت‌آمیز بودن نصب اطمینان حاصل کنید. با نصب Python، به جدیدترین ویژگی‌ها و بهبودهای عملکردی این زبان برنامه‌نویسی دسترسی خواهید داشت که به توسعه‌دهندگان کمک می‌کند تا کدهای بهینه‌تری بنویسند.



```
Python 3.11 (64-bit)
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

شکل ۳-۱: پایتون نصب شده

۳-۲-۲- ساخت Virtual Environment و نصب کتابخانه‌ها

برای جلوگیری از تداخل وابستگی‌ها بین پروژه‌های مختلف و ایجاد یک محیط ایزوله، از یک محیط مجازی (Virtual Environment) استفاده شده است. این عمل یک رویه استاندارد و بهترین تمرین در توسعه پایتون است. فایل gitignore. پروژه به طور مشخص دایرکتوری‌های رایج محیط مجازی مانند venv/ و venv. را نادیده می‌گیرد که این موضوع، استفاده از این رویکرد در فرآیند توسعه را تأیید می‌کند.

پس از ایجاد و فعال‌سازی محیط مجازی، تمام کتابخانه‌ها و بسته‌های نرم‌افزاری مورد نیاز پروژه با استفاده از فایل requirements.txt نصب می‌شوند. این کار با اجرای دستور `pip install -r requirements.txt` انجام می‌پذیرد.

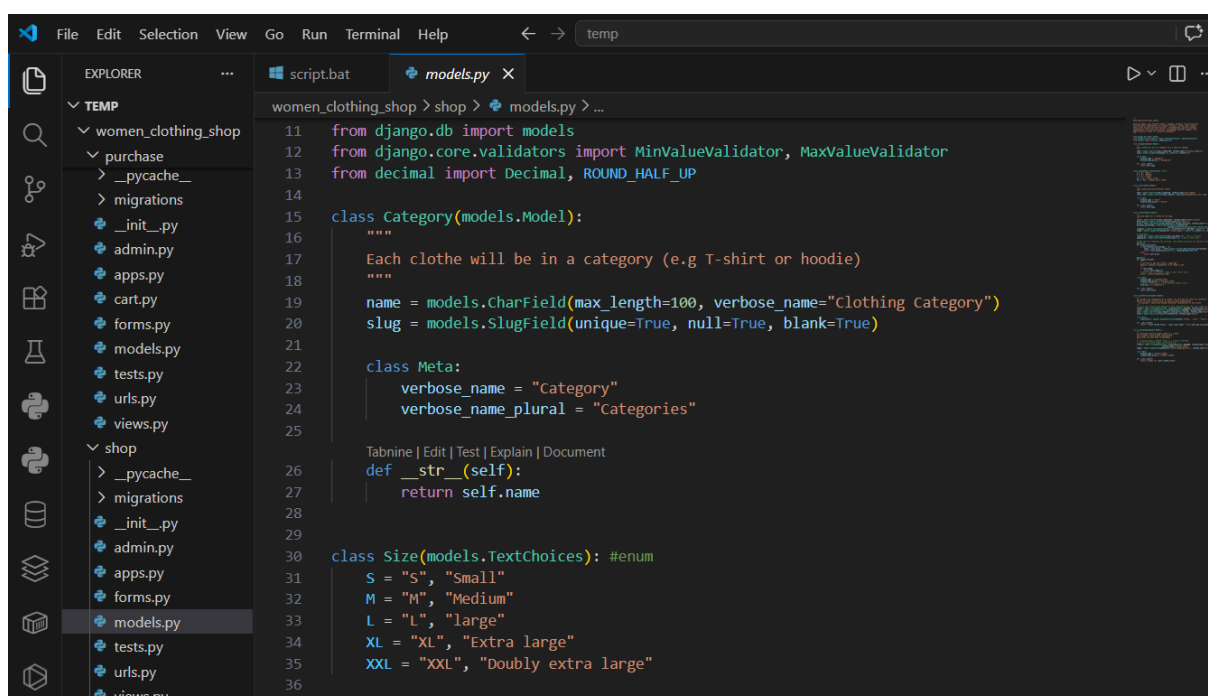
۳-۲-۳- استفاده از IDE مناسب مانند VSCode

- انتخاب یک محیط توسعه مناسب، یکی از عوامل مهم در افزایش بهره‌وری و کیفیت کدنویسی در پروژه‌های نرم‌افزاری است. استفاده از Visual Studio Code (VS Code) در یک پروژه جنگویی مانند "women-clothing-shop" مزایای متعددی دارد که به بهبود فرآیند توسعه کمک می‌کند:
- پشتیبانی مناسب از توسعه جنگو: VS Code با استفاده از افزونه‌های مرتبط، امکانات مناسبی برای توسعه پروژه‌های جنگو فراهم می‌کند. این ابزار امکان ویرایش و مدیریت فایل‌های مختلف پروژه از جمله مدل‌ها، ویوها، قالب‌ها و فایل‌های تنظیمات را به صورت منظم و سریع در اختیار توسعه‌دهنده قرار می‌دهد.
- ابزارهای خطایابی (Debugging): VS Code دارای ابزارهای دیباگ قدرتمندی است که امکان اجرای مرحله به مرحله کد، بررسی مقادیر متغیرها و شناسایی خطاها را فراهم می‌کند. این قابلیت به ویژه در بخش‌های حساس پروژه، مانند منطق ثبت سفارش یا پرداخت، بسیار کاربردی است.
- تکمیل خودکار کد هوشمند (Intelligent Code Completion): این محیط با بهره‌گیری از افزونه‌های برنامه‌نویسی، قابلیت تکمیل خودکار کد و ارائه پیشنهادهای هوشمند را فراهم می‌کند. این ویژگی باعث افزایش سرعت توسعه، کاهش خطاهای تایپی و بهبود دقت در نوشتن کد می‌شود.
- مدیریت وابستگی‌ها و محیط مجازی: VS Code به خوبی با محیط‌های مجازی Python و فایل requirements.txt هماهنگ می‌شود و اجرای دستورات پروژه و مدیریت وابستگی‌ها را ساده‌تر

می‌کند. همچنین ترمینال داخلی آن، اجرای دستورات مربوط به جنگو مانند runserver و makemigrations را آسان می‌سازد.

- سبک بودن و قابلیت شخصی‌سازی: یکی از مزیت‌های مهم VS Code سبک بودن و انعطاف‌پذیری بالای آن است. توسعه‌دهنده می‌تواند با نصب افزونه‌های موردنیاز، محیط کار را متناسب با نیازهای پروژه تنظیم کند.

در نتیجه، استفاده از VS Code در پروژه “women-clothing-shop” به سازمان‌دهی بهتر فایل‌ها، کاهش خطاهای برنامه‌نویسی، افزایش سرعت توسعه و بهبود مدیریت پروژه کمک کرده است.



شکل ۳-۲: برنامه‌ی VSCode

۳-۲-۴- رعایت Clean Code

در فرآیند پیاده‌سازی پروژه تلاش شده است تا اصول کدنویسی تمیز (Clean Code) رعایت شود تا کد نهایی خوانا، قابل نگهداری و توسعه‌پذیر باشد. این اصول در بخش‌های مختلفی از پروژه مشهود است:

- خوانایی و نام‌گذاری: برای متغیرها، توابع و کلاس‌ها از نام‌های توصیفی و معنادار استفاده شده است (مانند cart_add, clothe_edit, change_password) که درک عملکرد هر بخش را آسان می‌کند.

- اصل مسئولیت واحد (Single Responsibility): منطق برنامه به صورت ماژولار در فایل‌های مجزا سازماندهی شده است. برای مثال، برنامه در چهار ساختار مجزا (که در جنگو، اپلیکیشن نام دارند) تحت عنوان‌های accounts, core, purchase, shop تعریف شده است که باعث می‌شود هر بخش، کاملاً مسئولیت واحد داشته و برنامه اصلی، به چند زیربرنامه تبدیل شود. همچنین در هر اپلیکیشن، تمام تعاریف مدل‌ها در models.py، منطق درخواست و پاسخ در views.py، و فرم‌ها در forms.py قرار دارند.

۳-۲-۵- کامنت‌گذاری و مستندسازی

یکی از نقاط قوت پیاده‌سازی این پروژه، توجه ویژه به مستندسازی کد است. این مستندسازی به دو شکل اصلی انجام شده است تا درک و توسعه آینده کد را برای هر توسعه‌دهنده‌ای ممکن سازد:

- **Docstrings (رشته‌های توضیحات):** تقریباً تمام ماژول‌ها، کلاس‌ها و توابع اصلی پروژه دارای Docstring‌های چندخطی هستند. این توضیحات، هدف کلی، پارامترهای ورودی و گاهی خروجی هر بخش را تشریح می‌کنند و به عنوان مستندات داخلی کد عمل می‌نمایند.
- **کامنت‌های درون‌خطی (Inline Comments):** در بخش‌هایی از کد که منطق پیچیده‌تر یا غیر واضحی دارند، از کامنت‌های کوتاه برای روشن‌سازی عملکرد یک قطعه کد خاص استفاده شده است. این کامنت‌ها به درک جزئیات پیاده‌سازی کمک شایانی می‌کنند.

```

Tabnine | Edit | Test | Explain | Document
@staff_member_required
def clothe_edit(request, clothe_id):
    """
    Handles editing an existing clothe (Admin/Staff only).
    GET: Displays ClotheForm, ClotheImageFormSet and ClotheVariantFormSet pre-filled.
    POST: Validates and saves changes to clothe and its gallery and its variants.
    """
    if not request.user.is_staff:
        messages.error(request, "شما اجازه دسترسی به این صفحه را ندارید.")
        return redirect('home')

    clothe = get_object_or_404(Clothe, id=clothe_id)

    if request.method == 'POST':
        form = ClotheForm(request.POST, request.FILES, instance=clothe)
        # Initialize formset with POST data and the existing clothe
        variant_formset = ClotheVariantFormSet(request.POST, instance=clothe)
        image_formset = ClotheImageFormSet(request.POST, request.FILES, instance=clothe)

```

شکل ۳-۳: نمونه کامنت‌گذاری‌ها

۳-۳- Django در پروژه

فریم‌ورک جنگو به عنوان ستون فقرات و شالوده اصلی پروژه "women-clothing-shop" به کار گرفته شده است. این فریم‌ورک با ارائه یک ساختار منسجم و مجموعه‌ای از ابزارهای قدرتمند، فرآیند توسعه یک وب اپلیکیشن پیچیده مانند فروشگاه آنلاین پوشاک زنانه را به شکل چشمگیری ساده و بهینه کرده است. در ادامه، نحوه استفاده از قابلیت‌های مختلف جنگو در این پروژه به تفصیل شرح داده می‌شود.

۳-۳-۱- راه‌اندازی یک پروژه جدید در Django

پروژه "women-clothing-shop" با استفاده از دستور استاندارد جنگو `django-admin startproject women-clothing-shop` آغاز شده است. این دستور، ساختار دایرکتوری اولیه و فایل‌های پیکربندی اصلی پروژه را ایجاد می‌کند. فایل `manage.py` به عنوان ابزار خط فرمان برای تعامل با پروژه (مانند اجرای سرور توسعه و اعمال تغییرات پایگاه داده) عمل می‌کند. دایرکتوری داخلی `women-clothing-shop/` شامل فایل‌های حیاتی زیر است:

- `settings.py`: فایل مرکزی تنظیمات پروژه.
- `urls.py`: فایل اصلی مسیریابی URL های پروژه.
- `wsgi.py` و `asgi.py`: نقاط ورود برای وب‌سرورهای سازگار با WSGI و ASGI جهت استقرار پروژه.

۳-۳-۲- راه‌اندازی یک اپلیکیشن جدید در Django

منطق اصلی و قابلیت‌های کسب‌وکار پروژه "women-clothing-shop" در چهار اپلیکیشن به نام‌های `core`، `accounts`، `purchase` و `shop` قرار گرفته است و ساختاری ماژولار شامل فایل‌هایی مانند `models.py`، `views.py` و `forms.py` را به تفکیک هر اپلیکیشن فراهم کرده است. برای مثال، اپلیکیشن `core` با دستور `python manage.py startapp core` ایجاد می‌شود. پس از ایجاد، این اپلیکیشن باید به پروژه معرفی شود. این کار با افزودن نام آن (`core.apps.CoreConfig`) به لیست `INSTALLED_APPS` در فایل `settings.py` انجام شده است تا جنگو بتواند مدل‌ها، URL ها و سایر اجزای آن را شناسایی و استفاده کند. همین رویکرد، باید برای هر اپلیکیشن به صورت مجزا انجام شود.

۳-۳-۳- انجام تنظیمات مناسب برای پروژه در Django

فایل settings.py قلب تپنده پیکربندی پروژه است. در این پروژه، تنظیمات کلیدی زیر به دقت انجام شده است:

- **INSTALLED_APPS**: علاوه بر اپلیکیشن‌های پیش‌فرض جنگو، اپلیکیشن‌های سفارشی accounts, core, shop و purchase در این بخش ثبت شده‌اند.
- **DATABASES**: برای محیط توسعه، از پایگاه داده سبک و کارآمد SQLite استفاده شده است.
- **TEMPLATES**: مسیر دایرکتوری قالب‌های اصلی پروژه (templates) و همچنین پردازشگر زمینه (Context Processor) سفارشی core.context_processors.cart در این بخش تعریف شده‌اند.
- **AUTH_USER_MODEL**: به جنگو اعلام شده است که از مدل accounts.CustomUser به عنوان مدل پیش‌فرض برای مدیریت کاربران استفاده کند.
- **فایل‌های استاتیک و رسانه**: متغیرهای STATIC_URL/STATICFILES_DIRS و MEDIA_URL/MEDIA_ROOT برای مدیریت فایل‌های ثابت (CSS, JS) و فایل‌های بارگذاری شده توسط کاربر (تصاویر) پیکربندی شده‌اند.

۳-۳-۴- تعریف Model ها در Django

مدل‌ها (Models) در فریم‌ورک جنگو، قلب تپنده و بنیان ساختار داده‌ای هر اپلیکیشن هستند. آن‌ها به عنوان "تنها منبع حقیقت (Single Source of Truth)" برای داده‌های پروژه عمل کرده و ساختار جداول پایگاه داده، روابط بین آن‌ها و منطق مرتبط با داده‌ها را تعریف می‌کنند. در پروژه "women-clothing-shop"، مدل‌ها بین ۴ اپلیکیشن بر اساس ماهیت و کاربردشون تقسیم شدند و در فایل models.py موجود در دایرکتوری هر اپلیکیشن پیاده‌سازی شده‌اند. اپلیکیشن accounts مربوط به کاربران، اپلیکیشن purchase، مربوط به فرایند ثبت سفارش و خرید، اپلیکیشن shop، مربوط به محصولات و اپلیکیشن core، صرفاً شامل بخش‌های هسته‌ای و مشترک بین سایر اپلیکیشن‌ها است و مدلی در آن تعریف نشده است. این مدل‌ها با استفاده از ORM (Object-Relational Mapper) قدرتمند جنگو، کلاس‌های پایتون را به جداول پایگاه داده متصل می‌کنند و به توسعه‌دهنده اجازه می‌دهند تا بدون نوشتن مستقیم کوئری‌های SQL، با داده‌ها تعامل کنند.

در ادامه، هر یک از مدل‌های کلیدی این پروژه به تفکیک اپلیکیشن به تفصیل شرح داده می‌شوند:

۱. مدل‌های اپلیکیشن accounts

این اپلیکیشن صرفاً شامل مدل CustomUser است که برای مدیریت کاربران سیستم طراحی شده و با گسترش مدل کاربر پیش‌فرض جنگو، قابلیت‌های سفارشی مورد نیاز پروژه را فراهم می‌کند.

- وراثت: این کلاس از **AbstractUser** جنگو ارث‌بری می‌کند که تمام فیلدهای استاندارد کاربر مانند (username, password, email, first_name, last_name) را به همراه سیستم احراز هویت فراهم می‌کند.

- فیلدهای سفارشی:

○ **mobile_number**: یک **CharField** برای ذخیره شماره تلفن کاربر.

```
class CustomUser(AbstractUser):  
  
    """  
    Extends the default Django User model (AbstractUser).  
  
    This allows us to use Django's built-in authentication  
    while adding custom fields like a profile picture and phone number.  
    """  
    phone_number = models.CharField(max_length=15, null=True, blank=True, verbose_name="Phone Number")  
  
    Tabnine | Edit | Test | Explain | Document  
    def __str__(self):  
        """String representation, used in Admin panel and debugging."""  
        return self.username
```

شکل ۴-۳: پیاده‌سازی مدل CustomUser

۲. مدل‌های اپلیکیشن shop:

این اپلیکیشن، مدل‌های هسته اصلی بخش فروشگاه را تشکیل می‌دهد.

- **Category**: یک مدل ساده برای دسته‌بندی محصولات که شامل فیلدهای name (نام دسته‌بندی) و slug (شناسه قابل استفاده در URL) است.
- **Size**: یک enumeration که شامل یک پکیج از مقادیر ثابت به عنوان سایز لباس‌هاست.

- **Color**: یک مدل ساده که رنگ‌های مختلف موجود در پایگاه داده نگهداری می‌کند و شامل دو فیلد `name` (نام رنگ) و `color_code` (کد هگزادسیمال رنگ) است.
- **Clothe**: این مدل پیچیده، تمام اطلاعات مربوط به یک محصول لباس را در خود جای داده است:
 - **فیلدهای اصلی**: `title`, `description`, `price`, `discount_percentage`, `created_at`, `updated_at` و `image` (مسیردهی به آدرس `MEDIA_ROOT/clothe_images`).
 - **ارتباطات**: فیلد `category` از نوع **ForeignKey** است که هر لباس را به یک `Category` مرتبط می‌کند. آرگومان `on_delete=models.SET_NULL` تضمین می‌کند که با حذف یک دسته‌بندی، محصولات آن حذف نشده و صرفاً دسته‌بندی آن‌ها خالی می‌شود.
 - **قیمت‌گذاری**: فیلدهای `price` (قیمت اصلی) و `discount_percentage` (درصد تخفیف در صورت وجود) از نوع **DecimalField** برای دقت بالا در محاسبات مالی تعریف شده‌اند.
 - **Properties سفارشی**: یک `property` برای گرفتن مسیر عکس (`image_url`).
- **ClotheVariant**: این مدل، یک ترکیب یکتا از `Clothe`, `Size` و `Color` است و شامل فیلد `stock` (موجودی آن ترکیب) و فیلد `size` (که از `enum` انتخاب می‌شود) می‌باشد.
 - **ارتباطات**: فیلد `clothe` از نوع **ForeignKey** است که هر `variant` را به یک لباس ارتباط می‌دهد (هر `variant` به یک لباس مرتبط است ولی هر لباس می‌تواند شامل چند `variant` باشد) و فیلد `color` نیز از نوع **ForeignKey** است که هر `variant` را به یک رنگ مرتبط می‌کند.
- **ClotheImage**: این مدل، یک گالری از عکس‌های فرعی یک لباس می‌باشد که شامل یک فیلد `clothe` از نوع **ForeignKey** (به این معنا که هر لباس می‌تواند چند عکس داشته باشد ولی هر عکس برای یک لباس است) و یک فیلد `image` می‌باشد.

```

class Clothe(models.Model):
    """
    The main model for a clothe in the shop.
    """
    title = models.CharField(max_length=255, verbose_name="Product Title")
    description = models.TextField(verbose_name="Description")
    price = models.DecimalField(max_digits=12, decimal_places=0, verbose_name="Price")
    discount_percentage = models.SmallIntegerField(default=0,
        validators=[MinValueValidator(0), MaxValueValidator(100)], verbose_name="Discount Percentage")
    category = models.ForeignKey(Category, on_delete=models.SET_NULL, null=True, related_name="clothes", verbose_name="Category")
    image = models.ImageField(upload_to='clothe_images/', null=True, blank=True, verbose_name="Product Image")

    # Timestamps
    created_at = models.DateTimeField(auto_now_add=True) # Set on creation
    updated_at = models.DateTimeField(auto_now=True) # Set on every save

    # When the user completes the purchase, this method subtracts the discount from the base price (if there is a discount)
    @property
    def final_price(self):
        if self.discount_percentage > 0:
            final = self.price - ((self.price * Decimal(self.discount_percentage)) / Decimal('100'))
            return final.quantize(Decimal("1"), rounding=ROUND_HALF_UP)
        else:
            return self.price

    @property
    def image_url(self):
        """
        A safe way to get the clothe's image URL.
        Returns a default placeholder if no image is set.
        """
        if self.image:
            return self.image.url
        # Return path to a default image in your static files
        return '/static/img/default.png'

    class Meta:
        verbose_name = "Clothing Item"
        verbose_name_plural = "Clothing Items"
        # Default ordering for clothe queries (newest first)
        ordering = ['-created_at']

    def __str__(self):
        return self.title

```

شکل ۵-۳: پیاده‌سازی مدل Clothe

```

class ClotheVariant(models.Model):
    """
    We consider the combination of a clothe, its size and its color as a distinct
    entity in order to have stock for each of these combinations
    - we consider a same price between different combinations of one clothe
    """
    # one to many relationship because of each combination has only one clothe and color but each clothe or color can be related to many variants
    clothe = models.ForeignKey(Clothe, on_delete=models.CASCADE, related_name="variants")
    color = models.ForeignKey(Color, on_delete=models.CASCADE, verbose_name="Color")
    size = models.CharField(max_length=3, choices=Size.choices, verbose_name="Size")
    stock = models.PositiveIntegerField(default=0, verbose_name="Stock")

    class Meta:
        constraints = [models.UniqueConstraint(fields=['clothe', 'color', 'size'], name='unique_variant')]

    def __str__(self):
        return f"{self.clothe.title} - {self.color.name} - Size {self.get_size_display()}"

```

شکل ۶-۳: پیاده‌سازی مدل ClotheVariant

۳. مدل‌های اپلیکیشن purchase:

این اپلیکیشن شامل دو مدل است و سیستم خرید و سفارش پروژه را پیاده‌سازی می‌کند. این مدل‌ها برای ثبت و نگهداری تاریخچه سفارش‌های نهایی شده کاربران طراحی شده‌اند.

• Order:

- این مدل اطلاعات کلی یک سفارش مانند **user** سفارش‌دهنده، **total_price** نهایی، **shipping_address** و **status** سفارش (که از TextChoices برای تعریف وضعیت‌هایی مانند "در انتظار پرداخت"، "ارسال شده" و... استفاده می‌کند) را ذخیره می‌کند.

• OrderItem:

- این مدل نمایانگر یک "تصویر لحظه‌ای (Snapshot)" از هر محصول در یک سفارش است.
- فیلد **clotheVarinat** در این مدل با آرگومان **on_delete=models.SET_NULL** تعریف شده است.
- فیلد **price** قیمت محصول را دقیقاً در لحظه ثبت سفارش ذخیره می‌کند تا تغییرات آتی قیمت‌ها بر روی گزارش‌های مالی سفارش‌های قبلی تأثیری نگذارد.
- فیلد **quantity** که تعدادی که از یک **variant** ثبت سفارش شده است را نگهداری می‌کند و دقیقاً در لحظه ثبت سفارش ذخیره می‌شود.

• Cart:

- در این پروژه سبد خرید به‌صورت **session-based** پیاده‌سازی شده و جدول جداگانه در پایگاه داده ندارد بنابراین به صورت مدل هم تعریف نشده است.
- پیاده‌سازی در کلاس **purchase.cart.Cart** قرار دارد که داده‌های سبد را در **request.session** نگهداری می‌کند.

```

class Order(models.Model):
    """
    Represents a customer's order.
    This acts as a container for the OrderItems and stores
    customer info, shipping details, and status.
    """
    STATUS_CHOICES = (
        ('processing', 'Processing'),
        ('shipped', 'Shipped'),
        ('cancelled', 'Cancelled'),
    )

    # Use settings.AUTH_USER_MODEL for foreign keys to your User model.
    # This is Django's recommended best practice.
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='orders',
                             verbose_name="User")
    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Created At")
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='processing', verbose_name="Order Status")

    shipping_address = models.TextField(verbose_name="Shipping Address", blank=True)

    class Meta:
        verbose_name = "Order"
        verbose_name_plural = "Orders"
        ordering = ['-created_at']

    Tabnine | Edit | Test | Explain | Document
    def __str__(self):
        return f"Order {self.id} by {self.user.username}"

    Tabnine | Edit | Test | Explain | Document
    @property
    def total_price(self):
        """
        Calculates the total price of the order.
        It sums the 'total_price' of all related OrderItems.
        'self.items.all()' works because of 'related_name="items"'
        in the OrderItem model.
        """
        return sum((item.total_price for item in self.items.all()), Decimal('0'))

```

شکل ۷-۳: پیاده‌سازی مدل Order

```

class OrderItem(models.Model):
    """
    Represents a single "line item" within an Order.
    This links a specific Clothe to an Order, storing the
    quantity and price *at the time of purchase*.
    """
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name='items', verbose_name="Order")

    # on_delete=models.SET_NULL is important!
    # If a clothe is deleted, we don't want to delete the OrderItem.
    # This preserves the order history. The clothe field will just become Null.
    clothe_variant = models.ForeignKey(ClotheVariant, on_delete=models.PROTECT, null=True, related_name='order_items',
                                       verbose_name="Product")
    quantity = models.PositiveIntegerField(default=1, verbose_name="Quantity")

    # CRITICAL: Store the price at purchase time.
    # This prevents the order total from changing if the
    # Clothe.price is updated later.
    price = models.DecimalField(max_digits=12, decimal_places=0, verbose_name="Price (at purchase time)")

    class Meta:
        verbose_name = "Order Item"
        verbose_name_plural = "Order Items"

    Tabnine | Edit | Test | Explain | Document
    def __str__(self):
        # Check if clothe still exists
        clothe_name = self.clothe_variant.clothe.title if self.clothe_variant else "Deleted Product"
        return f"{self.quantity} x {clothe_name}"

    Tabnine | Edit | Test | Explain | Document
    @property
    def total_price(self):
        """Calculates the total for this specific line item."""
        return self.quantity * self.price

```

شکل ۸-۳: پیاده‌سازی مدل OrderItem

۳-۳-۵- تعریف و کاربرد Forms

فرم‌ها در فریم‌ورک جنگو یکی از حیاتی‌ترین ابزارها برای تعامل با کاربر هستند. آن‌ها به عنوان یک پل ارتباطی میان رابط کاربری (Frontend) و منطق پردازشی سرور (Backend) عمل کرده و سه وظیفه اصلی را بر عهده دارند: رندر کردن ویجت‌های HTML برای نمایش فیلدهای ورودی، اعتبارسنجی داده‌های ارسال شده توسط کاربر، و پردازش و پاک‌سازی این داده‌ها قبل از استفاده در منطق برنامه. در پروژه "women-clothing-shop"، فرم‌های مورد استفاده همانند مدل‌ها میان اپلیکیشن پخش شده‌اند و در هر اپلیکیشن (به جز core که فاقد فرم است) در فایل forms.py داخل دایرکتوری آن اپلیکیشن تعریف شده‌اند.

در ادامه، هر یک از فرم‌های کلیدی این پروژه به تفکیک اپلیکیشن‌ها به تفصیل شرح داده می‌شوند:

۱. فرم‌های اپلیکیشن accounts:

فرم‌های این اپلیکیشن برای فرایند ورود، ساخت حساب کاربری و یا ویرایش اطلاعات آن هستند.

- **فرم ورود کاربران:** برای این بخش فرم اختصاصی ساخته نشده و مستقیماً از AuthenticationForm جنگو استفاده شده که فرایند احراز هویت را به خوبی انجام می‌دهد.

شکل ۳-۹: فرم ورود کاربران

- **فرم ثبت‌نام کاربران (CustomUserCreationForm):** این فرم برای ایجاد حساب کاربری جدید طراحی شده و امنیت و صحت داده‌های اولیه کاربر را تضمین می‌کند.

- **نوع فرم:** این کلاس از **UserCreationForm** جنگو ارث‌بری می‌کند. این انتخاب هوشمندانه است، زیرا این کلاس به صورت پیش‌فرض منطق‌های مربوط به تأیید تکرار رمز عبور و هش کردن امن آن را پیاده‌سازی کرده است.

- **فیلدها:** شامل فیلدهای `username`, `email`, `password1` (رمز عبور) و `password2` (تکرار رمز عبور) است که همگی با برچسب‌ها و `placeholder` های فارسی برای بهبود تجربه کاربری سفارشی شده‌اند.

- **کلاس Meta:** این فرم به مدل `models.CustomUser` متصل شده است.

شکل ۱۰-۳: فرم ثبت‌نام کاربران (`CustomCreationForm`)

- **فرم ویرایش اطلاعات کاربران (`CustomUserEditForm`):** این فرم برای ویرایش اطلاعات حساب کاربر نظیر نام کاربری، ایمیل و شماره تلفن می‌باشد (گذرواژه را نمی‌توان در این فرم آپدیت کرد)

- **نوع فرم:** این کلاس از **UserChangeForm** جنگو ارث‌بری می‌کند.

- **کلاس Meta:** فقط فیلدهای `username`, `email` و `phone_number` قرار گرفته‌اند.

- گذرواژه: مستقیماً توسط **PasswordChangeForm** جنگو انجام می‌شود.



شکل ۱۱-۳: فرم ویرایش اطلاعات کاربران (**CustomUserEditForm**)

- فرم ویرایش اطلاعات ادمین (**AdminUserEditForm**): این فرم برای ویرایش اطلاعات حساب کاربر توسط ادمین می‌باشد. (ادمین توانایی ویرایش تمام کاربران را دارد)
 - نوع فرم: این کلاس نیز از **UserChangeForm** جنگو ارث‌بری می‌کند.
 - گذرواژه: مستقیماً توسط **PasswordChangeForm** جنگو انجام می‌شود.
 - کلاس **Meta**: علاوه بر موارد لحاظ شده در فرم **CustomUserEditForm**، در این فرم، فیلدهای **is_active**, **is_staff** و **is_superuser** نیز قرار گرفته‌اند.

ویرایش کاربر

اطلاعات کاربر را ویرایش کنید و ذخیره بزنید

نام کاربری:

admin

الزامی. 150 کاراکتر یا کمتر. فقط شامل حروف، اعداد، و علامات @/./+/-/.

آدرس ایمیل:

admin@gmail.com

:Phone Number

1234

فعال: ☒

نشان می‌دهد که آیا این کاربر اجازه فعالیت دارد یا خیر. به جای حذف کاربر این تیک را بردارید.

وضعیت کارمندی: ☒

نشان می‌دهد که آیا این کاربر می‌تواند وارد این وبگاه مدیریت شود یا خیر.

ابَرکاربر: ☒

نشان می‌دهد که این کاربر همه اجازه‌ها را دارد بدون آنکه به صراحت به او اختصاص داده شده باشد.

بازگشت

ذخیره

شکل ۱۲-۳: فرم ویرایش اطلاعات ادمین (AdminUserEditForm)

۲. فرم‌های اپلیکیشن purchase

- فرم ایجاد سفارش (OrderCreateForm): این فرم اطلاعات پستی را هنگام نهایی کردن یک سبد خرید ثبت می‌کند.
 - نوع فرم: این کلاس نیز از forms.ModelForm جنگو ارث‌بری می‌کند.
 - فیلد: شامل فیلد shipping_address است که آدرس پستی را ذخیره می‌کند.
- فرم به‌روزرسانی وضعیت سفارش (OrderStatusUpdateForm): این فرم در سطح دسترسی ادمین قرار دارد که می‌تواند وضعیت سفارش را آپدیت کند.

شکل ۱۳-۳: ایجاد سفارش (OrderCreateForm)

۳. فرم‌های اپلیکیشن shop:

- فرم ایجاد و ویرایش محصول (ClotheForm): این فرم اطلاعات پستی را هنگام نهایی کردن یک سبد خرید ثبت می‌کند.
 - نوع فرم: این کلاس نیز از forms.ModelForm جنگو ارث‌بری می‌کند.
 - فیلد: شامل فیلدهای title, description, price, discount_percentage, category و image است.
 - از Textarea برای توضیحات و FileInput برای بارگذاری تصویر استفاده می‌کند تا رابط کاربری اداری ساده و کارآمد باشد.
- فرم واریانت (ترکیب/تنوع) هر محصول (ClotheVariantForm): این فرم رنگ، سایز و موجودی هر ترکیب را مدیریت می‌کند.
 - برای هر محصول، واریانت‌هایی مانند ترکیب color + size + stock تعریف می‌کند تا مدیریت موجودی دقیق‌تر انجام شود.

○ اعتبارسنجی `clean_stock` تضمین می‌کند که موجودی منفی وارد نشود و مقدار صحیح حفظ گردد.

• **فرم تعریف دسته‌بندی محصولات (CategoryForm):** این فرم ساده، دسته‌بندی جدید برای لباس‌ها ایجاد می‌کند.

○ تنها فیلد `name` را دارد و با ویجت `TextInput` نمایشی منظم برای وارد کردن نام دسته‌بندی فراهم می‌کند.

• **فرم تعریف رنگ (ColorForm):** این فرم ساده، نوع رنگ برای لباس‌ها ایجاد می‌کند.

○ فیلدهای `name` و `color_code` را نمایش می‌دهد.

○ از ویجت `type="color"` برای انتخاب سریع رنگ استفاده می‌شود.

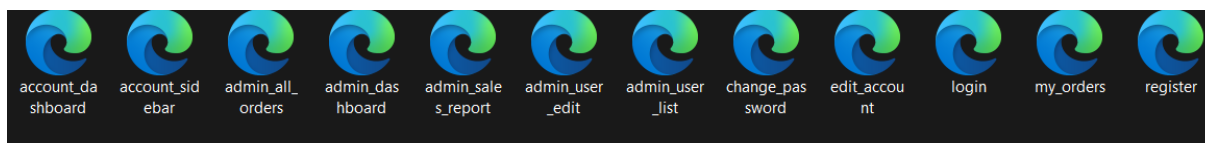
۶-۳-۳- تعریف ساختار Templates

قالب‌ها (Templates) مسئولیت لایه نمایش پروژه را بر عهده دارند. در این پروژه، یک دایرکتوری اصلی به نام `templates` در ریشه پروژه ایجاد شده و در فایل `settings.py` به جنگو معرفی شده است و سپس در درون این دایرکتوری، برای هر اپلیکیشن، دایرکتوری مجزایی شکل گرفته که هر کدام شامل قالب‌های خودشان می‌باشند. جنگو با استفاده از موتور قالب خود، این فایل‌های `HTML` را با داده‌های پویایی که از `View` ها ارسال می‌شوند، پردازش و رندر می‌کند. مفاهیمی مانند وراثت قالب (Template Inheritance) برای ایجاد یک ساختار پایه و گسترش آن در صفحات دیگر، و استفاده از تگ‌ها و فیلترهای جنگو برای نمایش منطق شرطی و قالب‌بندی داده‌ها، در این لایه به کار گرفته شده است.

در `Django`، فایل‌های `template` معمولاً در داخل پوشه‌ای به نام `templates` قرار می‌گیرند که می‌تواند شامل زیرپوشه‌های مختلفی برای هر اپلیکیشن باشد. به‌طور کلی، ساختار فولدر `templates` در این پروژه به صورت زیر می‌باشد:

```
women_clothing_shop/templates/accounts
women_clothing_shop/templates/core
women_clothing_shop/templates/purchase
women_clothing_shop/templates/shop
```

این ساختار می تواند شامل فایل های HTML متنوعی برای صفحات مختلف وبسایت باشد.



شکل ۱۴-۳: تمپلیت های استفاده شده در اپلیکیشن accounts

۳-۳-۷- ارتباط Templates با Backend

ارتباط میان بک اند (منطق پایتون) و فرانت اند (قالب های HTML) در این پروژه از دو طریق اصلی برقرار می شود:

۱. ارسال مستقیم Context از Views: در اکثر نماها، داده های مورد نیاز قالب از طریق یک دیکشنری به نام context به تابع render ارسال می شود. برای مثال، در `clothe_detail`، شیء محصول مورد نظر به قالب فرستاده می شود تا جزئیات آن نمایش داده شود.

۲. استفاده از Context Processors: برای داده هایی که باید در تمام صفحات در دسترس باشند (مانند تعداد اقلام سبد خرید)، یک پردازشگر زمینه سفارشی به نام `global_context` در فایل `context_processors.py` ایجاد شده است. این تابع، داده های مورد نظر را به طور خودکار به context تمام قالب ها اضافه می کند و از تکرار کد در نماهای مختلف جلوگیری می کند.

۳-۳-۸- مسیرها (URLs) و مدیریت آنها

سیستم مسیریابی URL در جنگو مسئولیت هدایت هر درخواست HTTP به View مربوطه را بر عهده دارد. در پروژه "women-clothing-shop" از یک ساختار دو لایه ای استفاده شده است:

۱. فایل `women-clothing-shop/urls.py` به عنوان نقطه ورود اصلی عمل می کند. این فایل مسیرهای سطح بالا مانند پنل ادمین (`admin/`) را تعریف کرده و سایر درخواست ها را به اپلیکیشن های دیگر واگذار می کند. برای مثال، با استفاده از تابع `include('core.urls')` به اپلیکیشن `core` متصل می شود.

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path('', include('core.urls')),
    path('', include('accounts.urls')),
    path('', include('shop.urls')),
    path('', include('purchase.urls')),
]
```

شکل ۱۵-۳ فایل urls.py

۲. فایل core/urls.py شامل الگوهای URL مربوط به اپلیکیشن اصلی است (صفحه اصلی و مسیر about/). در این فایل به یک تابع View مشخص در فایل core/views.py متصل شده است. برای بقیه اپلیکیشن‌ها هم همین رویکرد انجام شده است و مسیرهای مربوطه هر اپلیکیشن View های موجود در همان اپلیکیشن متصل شده‌اند.

```
urlpatterns = [
    # --- Authentication URLs ---
    path('register/', views.register, name='register'),
    path('login/', views.login_user, name='login'),
    path('logout/', views.logout_user, name='logout'),
    path('manage/login/', views.admin_login, name='admin-login'),
    path('manage/', views.admin_dashboard, name='admin-dashboard'),

    # --- User Account URLs ---
    path('account/', views.user_dashboard, name='user-dashboard'),
    path('account/edit/', views.edit_account, name='edit-account'),
    path('account/password/', views.change_password, name='change-password'),
    path('account/orders/', views.my_orders, name='my-orders'),

    # --- Admin/Management URLs ---

    # Product Management
    path('manage/product/add/', clothe_add, name='clothe-add'),
    path('manage/product/edit/<int:clothe_id>/', clothe_edit, name='clothe-edit'),
    path('manage/product/delete/<int:clothe_id>/', clothe_delete, name='clothe-delete'),
    path("manage/quick-add/category/", quick_add_category, name="quick-add-category"),
    path("manage/quick-add/color/", quick_add_color, name="quick-add-color"),
]
```

شکل ۱۶-۳ بخشی از فایل accounts/urls.py

۳-۳-۹- تعریف Commands (دستورات سفارشی)

فریم‌ورک جنگو امکان ایجاد دستورات مدیریتی سفارشی را فراهم می‌کند که می‌توان آن‌ها را از طریق خط فرمان با `python manage.py <command_name>` اجرا کرد. در این پروژه، دو دستور سفارشی بسیار کاربردی برای تسهیل فرآیند توسعه پیاده‌سازی شده است:

۱. `create_admin`: این دستور، یک ادمین را با اطلاعات از پیش تعریف شده و رمز عبور هش شده در پایگاه داده ایجاد می‌کند. (username = admin, password = admin۱۲۳)

۲. `seed_products`: این دستور ابتدا تمام محصولات و دسته‌بندی‌های قبلی را حذف کرده و سپس پایگاه داده را با ۲۰ محصول نمونه، همراه با جزئیات کامل، قیمت، موجودی و دسته‌بندی، پر می‌کند. این دستورات راه‌اندازی یک محیط توسعه جدید را بسیار سریع و آسان می‌کنند.

```
Command(BaseCommand):
    lp = "Seed clothes from data/products.json and copy images into MEDIA_ROOT."

    Tabnine | Edit | Test | Explain | Document
    def add_arguments(self, parser):
        parser.add_argument("--force", action="store_true", help="Upsert by title (update or insert if not exists)")
        parser.add_argument("--clear", action="store_true", help="Delete all clothes (and categories) from the database")

    Tabnine | Edit | Test | Explain | Document
    def handle(self, *args, **options):
        force = options["force"]
        clear = options["clear"]

        json_path = Path(settings.BASE_DIR) / "data" / "products.json"
        source_dir = Path(settings.BASE_DIR) / "static" / "clothe_images"
        media_root = Path(settings.MEDIA_ROOT)

        if not json_path.exists():
            self.stdout.write(self.style.ERROR(f"Missing JSON: {json_path}"))
            return

        if not source_dir.is_dir():
            self.stdout.write(self.style.ERROR(f"Missing image source dir: {source_dir}"))
            return

        if clear:
            self.stdout.write(self.style.WARNING("Clearing Clothe data..."))
            Cloth.objects.all().delete()

        data = json.loads(json_path.read_text(encoding="utf-8"))
        products = data.get("products") or []
        if not products:
            self.stdout.write(self.style.WARNING("No products found under key 'products'."))
            return
```

شکل ۱۷-۳: بخشی از پیاده‌سازی یک دستور سفارشی

۳-۳-۱۰- تعریف دایرکتوری Media

دایرکتوری media برای ذخیره‌سازی فایل‌هایی که توسط کاربران بارگذاری می‌شوند، مانند عکس محصولات، در نظر گرفته شده است. در فایل `settings.py`، متغیر `MEDIA_ROOT` مسیر فیزیکی این دایرکتوری بر روی سرور را مشخص می‌کند، در حالی که `MEDIA_URL` پیشوند URL عمومی برای دسترسی به این فایل‌ها از طریق مرورگر است. در مدل `Clothe`، فیلد `Image` با استفاده از آرگومان `upload_to` به جنگو دستور می‌دهد که فایل‌های بارگذاری شده را در زیرشاخه‌های مشخصی از این دایرکتوری ذخیره کند.

۳-۳-۱۱- تعریف دایرکتوری Static

دایرکتوری static محل نگهداری فایل‌های ثابت و ایستا پروژه است که شامل فایل‌های CSS، JavaScript، فونت‌ها و تصاویر عمومی وب‌سایت (مانند لوگو) می‌شود. در فایل settings.py، متغیر STATICFILES_DIRS به جنگو اعلام می‌کند که در دایرکتوری اصلی static در ریشه پروژه به دنبال این فایل‌ها بگردد. متغیر STATIC_URL نیز پیشنهاد URL برای فراخوانی این فایل‌ها در صفحات HTML است.

۳-۳-۱۲- Admin در Django

پروژه women-clothing-shop از پنل مدیریت پیش‌فرض جنگو استفاده می‌کند، اما بخش ادمین با تنظیمات سفارشی روی مدل‌ها و یک داشبورد داخلی برای کاربران دارای دسترسی مدیریت بهبود یافته است.

- صفحه‌ی داشبورد مدیریت (admin_dashboard در accounts/views.py):

- برای کاربران staff طراحی شده و اطلاعات کلی کسب‌وکار را نشان می‌دهد.
- بررسی وضعیت سفارشات.
- آمارهایی مانند تعداد محصولات، تعداد سفارش‌ها، سفارش‌های در حال پردازش و تعداد کاربران را می‌آورد.

- سفارشی‌سازی مدل‌ها در پنل ادمین:

- در accounts/admin.py مدل CustomUser با list_display، list_filter و search_fields قابل جستجو و مرتب‌سازی شده است.
- در purchase/admin.py مدل Order با OrderItemInline نمایش داده می‌شود تا اقلام هر سفارش مستقیم در صفحه سفارش قابل مشاهده باشند.
- همچنین OrderAdmin از list_display، list_filter، search_fields و date_hierarchy برای بهبود مدیریت سفارش‌ها بهره می‌برد.

۳-۴- طراحی صفحات با HTML و CSS و Javascript

لایه نمایش (Frontend) در پروژه "women-clothing-shop"، نقطه‌نهایی تعامل کاربر با سیستم است و طراحی آن با استفاده از سه فناوری استاندارد وب یعنی HTML، CSS و JavaScript صورت گرفته است. جنگو به عنوان فریم‌ورک بک‌اند، وظیفه مدیریت، پردازش و ارائه این فایل‌ها به مرورگر کاربر را بر

عهده دارد. ساختار فایل‌ها و نحوه یکپارچگی آن‌ها با بک‌اند به گونه‌ای طراحی شده است که هم قابلیت نگهداری بالایی داشته باشد و هم تجربه کاربری مدرن و پویایی را فراهم کند.

۱. HTML و موتور قالب جنگو (Django Template Engine)

زبان نشانه‌گذاری **HTML (HyperText Markup Language)** اسکلت و ساختار اصلی تمام صفحات وب سایت "women-clothing-shop" را تشکیل می‌دهد. در این پروژه، به جای استفاده از فایل‌های HTML ایستا، از موتور قالب جنگو (**Django Template Language - DTL**) بهره گرفته شده است. این رویکرد به ما اجازه می‌دهد تا صفحات HTML را به صورت پویا و با تزریق داده‌های ارسال شده از بک‌اند، تولید کنیم.

- **ساختار و محل قرارگیری فایل‌ها:** فایل‌های قالب پروژه در دو مکان اصلی سازماندهی شده‌اند که این ساختار در فایل `settings.py` پیکربندی شده است:
 - یک دایرکتوری اصلی به نام `templates` در ریشه پروژه (در مسیر `women-clothing-shop/templates/`) قرار دارد که شامل زیردایرکتوری‌های `accounts/`, `core/`, `purchase/` و `shop/` می‌باشد و هر کدام، قالب‌های مربوط به خود را دارند. برای مثال، در `women-clothing-shop/templates/core/` قالب `base.html` وجود دارد که بین همه اپلیکیشن‌ها مشترک بوده و تمام قالب‌ها آن را به ارث می‌برند.
- **کاربرد در پروژه:** از طریق DTL، داده‌هایی که از `View` ها به قالب‌ها ارسال می‌شوند (مانند لیست محصولات، اطلاعات کاربر وارد شده یا جزئیات یک سفارش) با استفاده از سینتکس‌هایی مانند `{{` `clothe.title` نمایش داده می‌شوند. همچنین، با استفاده از تگ‌های منطقی مانند `{% for clothe` `in clothes` `%}` و `{% if user.is_authenticated` `%}`، ساختارهای تکرارشونده و محتوای شرطی در صفحات ایجاد می‌گردد.

۲. CSS و استایل‌دهی (Styling)

زبان **CSS (Cascading Style Sheets)** مسئولیت کامل ظاهر بصری، استایل‌دهی، رنگ‌بندی و چیدمان (Layout) تمام المان‌های وب سایت "women-clothing-shop" را بر عهده دارد. این فایل‌ها به صفحات HTML هویت و زیبایی می‌بخشند و تجربه بصری یکپارچه‌ای را برای کاربر ایجاد می‌کنند.

- ساختار و محل قرارگیری فایل‌ها: فایل CSS پروژه، به عنوان فایل استاتیک، در دایرکتوری اصلی static واقع در ریشه پروژه (مسیر /static/css/ women-clothing-shop) نگهداری می‌شود. این مسیر در فایل settings.py از طریق متغیر STATICFILES_DIRS به جنگو معرفی شده است.

۳. JavaScript و تعاملات پویا (Dynamic Interactions)

زبان JavaScript برای افزودن قابلیت‌های تعاملی، پویا و مدرن به رابط کاربری پروژه "women-clothing-shop" به کار رفته است. این زبان به ما اجازه می‌دهد تا بدون نیاز به بارگذاری مجدد کامل صفحه، با سرور ارتباط برقرار کرده و تغییراتی را در محتوای صفحه ایجاد کنیم، که این امر منجر به بهبود چشمگیر تجربه کاربری (UX) می‌شود.

- ساختار و محل قرارگیری فایل‌ها: فایل‌های JavaScript نیز همانند فایل‌های CSS، در دایرکتوری static در ریشه پروژه (مسیر /static/js/ women-clothing-shop) قرار دارند.
- کاربرد در پروژه: استفاده از JavaScript در این پروژه در دو بخش کلیدی قابل مشاهده است:

۱. ارتباطات غیرهمزمان (AJAX): مهم‌ترین کاربرد JavaScript در این پروژه، مدیریت سبد خرید است. توابع View مربوط به افزودن (add_to_cart)، ویرایش (update_cart) و حذف (remove_from_cart) محصول از سبد خرید، به جای رندر کردن یک صفحه HTML کامل، یک پاسخ با فرمت JSON برمی‌گردانند. این موضوع نشان می‌دهد که کدهای JavaScript در فرانت‌اند، درخواست‌های AJAX را به این اندپوینت‌ها ارسال کرده و پس از دریافت پاسخ، بخش‌های مربوط به سبد خرید در صفحه را به صورت پویا و بدون نیاز به رفرش، به‌روزرسانی می‌کنند.

۳-۵- پایگاه داده SQL در پروژه

پایگاه داده به عنوان مخزن اصلی و پایدار داده‌ها، نقشی حیاتی در عملکرد سیستم "women-clothing-shop" ایفا می‌کند. تمام اطلاعات مربوط به کاربران، محصولات، سفارش‌ها و سایر موجودیت‌های سیستم در این پایگاه داده ذخیره و مدیریت می‌شود. در این پروژه، از سیستم مدیریت ارتباط با پایگاه داده (ORM) قدرتمند جنگو استفاده شده است که به ما اجازه می‌دهد تا ساختار پایگاه داده را از طریق تعریف کلاس‌های پایتون در فایل‌های models.py موجود در دایرکتوری‌های هر اپلیکیشن، مدل‌سازی کنیم. جنگو سپس به صورت خودکار این مدل‌ها را به جداول SQL و روابط بین آن‌ها در پایگاه داده منتخب (در فاز توسعه، SQLite) ترجمه می‌کند.

۳-۵-۱- ساختار پایگاه داده و ارتباط جداول

پروژه‌ی women Clothing Shop از مجموعه‌ای از مدل‌های Django تشکیل شده که هر کدام به صورت جدول در پایگاه داده منتقل می‌شوند. نام جدول‌های واقعی بر اساس قالب Django به صورت appname_modelname خواهد بود (مثلاً shop_clothe, purchase_order). روابط بین جداول از طریق کلیدهای خارجی (ForeignKey) و محدودیت‌های یکپارچگی (مانند UniqueConstraint و select_for_update برای قفل‌گذاری در تراکنش‌ها) برقرار شده‌اند. شرح جداول و فیلدهای کلیدی به صورت خلاصه و منطبق با ساختار پروژه در ادامه آورده شده است.

۱. جدول accounts_customuser

- مسئول نگهداری اطلاعات کاربران است و از AbstractUser جنگو گسترش یافته است.
- نقش‌ها و سطوح دسترسی با استفاده از فیلدهای کاربری و گروه‌ها/پرمیژن‌ها مدیریت می‌شوند (در کد پروژه سطح ادمین با is_staff/is_superuser مشخص می‌شود).
- فیلدهای مهم:

- id, username, password (هش شده)، email, first_name, last_name
- phone_number (فیلد دلخواه برای شماره تلفن)
- is_active, is_staff, is_superuser: فیلدهای بولی

۲. جدول shop_category

- برای دسته‌بندی محصولات استفاده می‌شود.
 - رابطه: یک به چند با جدول محصولات (shop_clothe).
 - فیلدهای مهم:
- id (PK)، خود مدل فعلی در پروژه از AutoField استفاده می‌کند
 - name (نام دسته)، slug (شناسه متنی مناسب برای URL)

۳. جدول shop_clothe

- نمایانگر محصول اصلی (پوشاک) است.

- ویژگی‌ها: منطق محاسبه قیمت نهایی (final_price) به صورت property در مدل پیاده‌سازی شده و هنگام ایجاد OrderItem قیمت نهایی ذخیره می‌گردد تا تغییرات بعدی قیمت روی سفارش‌های تاریخی اثر نگذارد.

- فیلدهای مهم:

- id, title (نام محصول), description
- image (فایل تصویر/مسیر)
- price (قیمت پایه), discount_percentage (درصد تخفیف)
- category_id (ForeignKey به shop_category, در پروژه رفتار
- on_delete=models.SET_NULL برای نگهداری محصول در صورت حذف دسته‌بندی تعریف شده است)
- created_at, updated_at: مهرهای زمانی:

۴. جدول shop_color و enum Size

- shop_color نگهدارنده رنگ‌های تعریف شده است:
- فیلدها: id, name
- color_code (کد هگز رنگ)
- Size در مدل به صورت TextChoices تعریف شده و برای فیلد ClotheVariant.size استفاده می‌شود (مقادیر S, M, L, XL, ...).

۵. جدول shop_clothevariant

- هر ترکیب (محصول + رنگ + سایز) به عنوان یک واریانت مجزا مدل شده است تا موجودی مستقل برای هر ترکیب نگهداری شود.
- فیلدهای مهم: clothe_id (ForeignKey به shop_clothe), color_id (FK به shop_color), size (از Size), stock (موجودی عددی)
- محدودیت‌ها: UniqueConstraint(fields=['clothe', 'color', 'size'], name='unique_variant') برای جلوگیری از تکرار همان ترکیب.

- قفل گذاری با select_for_update() در هنگام ثبت سفارش برای جلوگیری از oversell استفاده می شود.

۶. جدول shop_clotheimage

- تصاویر گالری مربوط به هر محصول را نگهداری می کند.
- فیلدها: image_id, clothe_id (FK) (فایل)
- رفتار حذف: on_delete=models.CASCADE برای حذف تصاویر مرتبط در صورت حذف محصول.

۷. جدول purchase_order

- نمایانگر سفارش های نهایی شده است.
- فیلدهای مهم: id (PK), user_id (FK به accounts_customuser), created_at, status (مثلاً processing, shipped, cancelled), shipping_address
- مجموع قیمت نهایی سفارش از جمع OrderItem.total_price محاسبه می شود (در مدل property total_price وجود دارد).
- تراکنش ها: ایجاد سفارش داخل transaction.atomic() و با قفل گذاری واریانت ها انجام می شود تا یکپارچگی موجودی تضمین گردد.

۸. جدول purchase_orderitem

- هر رکورد نمایانگر یک خط سفارش است و اطلاعات هر آیتم خرید را نگهداری می کند.
- فیلدهای مهم: order_id (FK به purchase_order), clothe_variant_id (FK به shop_clothevariant), quantity, price (قیمت ثبت شده در زمان خرید)
- نکته مهم: قیمت (price) در زمان ثبت سفارش ذخیره می شود تا تغییرات بعدی قیمت محصول در سفارش های قبلی تأثیر نگذارد.

۹. خلاصه و نکات یکپارچه‌سازی

- روابط اصلی بین جداول:

shop_clothe ← (FK) shop_category ○

shop_clothevariant ← (FK) shop_clothe, shop_color ○

shop_clotheimage ← (FK) shop_clothe ○

purchase_order ← (FK) accounts_customuser ○

purchase_orderitem ← (FK) purchase_order, shop_clothevariant ○

- سبد خرید در پروژه به صورت نشست (session) مدیریت می شود و جدول پایگاه داده جدا برای آن وجود ندارد.

فصل چهارم

نصب و راهنمای پروژه

۴-۱- نحوه نصب و راه اندازی پروژه

برای راه اندازی یک پروژه Django، ابتدا باید از نصب پایتون بر روی سیستم خود مطمئن شویم. پایتون باید به روز باشد تا از تمامی قابلیت های جدید و بهینه سازی ها بهره مند شویم. می توانیم با اجرای دستور `python -version` در ترمینال یا Command Prompt، نسخه نصب شده را بررسی کنیم.

۴-۱-۱- ساخت محیط مجازی

پس از تأیید نصب پایتون، مرحله بعدی ایجاد یک محیط مجازی است. محیط های مجازی به ما این امکان را می دهند که وابستگی های پروژه های مختلف را به طور جداگانه مدیریت کنیم و از تداخل آن ها جلوگیری کنیم. برای ایجاد یک محیط مجازی، می توانیم از دستور زیر استفاده کنیم:

```
python -m venv .venv
```

در اینجا `.venv` نام محیط مجازی است که می توانیم آن را به دلخواه تغییر دهیم. پس از ایجاد محیط مجازی، باید آن را فعال کنیم. در ویندوز، با استفاده از:

```
.venv/Scripts/activate
```

۴-۱-۲- نصب کتابخانه های مورد نیاز

پس از فعال سازی محیط مجازی، نوبت به نصب کتابخانه های مورد نیاز پروژه می رسد. معمولاً این کتابخانه ها در فایل به نام `requirements.txt` قرار دارند. برای نصب آن ها، می توانیم از دستور زیر استفاده کنیم:

```
pip install -r requirements.txt
```

این دستور تمام کتابخانه های ذکر شده در فایل `requirements.txt` را نصب خواهد کرد.

۴-۱-۳- انجام مایگریشن‌ها و ایجاد داده‌های پیش‌فرض

پس از نصب کتابخانه‌ها، باید دستورات مربوط به مایگریشن را اجرا کنیم تا پایگاه داده پروژه آماده شود. ابتدا با اجرای دستور زیر، فایل‌های مایگریشن را ایجاد می‌کنیم:

```
python manage.py makemigrations
```

سپس با اجرای دستور زیر، مایگریشن‌ها را به پایگاه داده اعمال می‌کنیم:

```
python manage.py migrate
```

در مرحله بعد، ممکن است بخواهیم اطلاعات کاربران و محصولات پیش‌فرض را در پایگاه داده ایجاد کنیم. برای این کار می‌توانیم دستورات سفارشی مانند `create_admin` و `create_products` را تعریف کرده و اجرا کنیم. این دستورات معمولاً در فایل‌های مدیریت (`management commands`) قرار دارند.

۴-۱-۴- راه‌اندازی سرور توسعه

بعد از انجام مراحل بالا، اکنون می‌توانیم سرور توسعه `Django` را با استفاده از دستور زیر راه‌اندازی کنیم:

```
python manage.py runserver
```

این دستور سرور محلی را راه‌اندازی کرده و به ما این امکان را می‌دهد که پروژه خود را در مرورگر مشاهده کنیم.

۴-۱-۵- حذف فایل SQLite

در صورتی که بخواهیم پایگاه داده `SQLite` را دوباره بسازیم، می‌توانیم فایل `SQLite` موجود را حذف کنیم. این فایل معمولاً به نام `db.sqlite3` است. با حذف این فایل و اجرای دوباره دستور `migrate`، پایگاه داده جدیدی ایجاد خواهد شد که شامل تمامی جداول و ساختارهای مورد نیاز خواهد بود:

```
rm db.sqlite3 یا del db.sqlite3
```

```
python manage.py migrate
```

با انجام این مراحل، پروژه `Django` ما به‌طور کامل راه‌اندازی شده و آماده توسعه و گسترش است.

۴-۱-۶- استفاده از script.bat برای راحتی کار

برای راحتی و سرعت در تنظیمات و اجرای پروژه Django، می‌توانید از یک اسکریپت بت (script.bat) استفاده کنید. این اسکریپت تمام مراحل آماده‌سازی محیط و راه‌اندازی پروژه را به‌صورت خودکار انجام می‌دهد، به‌طوری‌که شما تنها با انتخاب گزینه‌های مورد نظر از منو، مراحل مختلف را طی می‌کنید. اسکریپت script.bat با گزینه‌های مختلف به شما کمک می‌کند تا محیط پروژه را آماده کنید، Django را پیکربندی کنید، یا پروژه را اجرا کنید. این اسکریپت دارای یک منوی ساده است که به شما اجازه می‌دهد از طریق انتخاب گزینه‌ها مراحل مختلف پروژه را به راحتی انجام دهید.

- گزینه صفر: پاک کردن تمام دیتاهای قبلی
- گزینه ۱: آماده‌سازی محیط Python
- گزینه ۲: آماده‌سازی Django
- گزینه ۳: اجرای پروژه
- گزینه ۴: خروج از اسکریپت

این اسکریپت به‌ویژه برای افرادی که مایلند به‌طور مکرر پروژه خود را پیکربندی یا اجرا کنند، مفید است و روند انجام کارها را تسهیل می‌کند.

```
=====
RESET/SETUP
=====
0. HARD RESET (Delete venv, DB, migrations, media)
1. Prepare Environment (venv & requirements)
2. Prepare Django (migrate, seed, create admin, collectstatic)

=====
COMMON
=====
3. Run Project (runserver)
4. Exit
=====
```

شکل ۱-۴: نمای script.bat

۴-۲- راهنمای سطوح دسترسی کاربران

سیستم فروشگاه آنلاین پوشاک زنانه "women-clothing-shop" برای مدیریت بهینه، امنیت و ارائه تجربه کاربری متناسب، یک ساختار کنترل دسترسی مبتنی بر نقش (Role-Based Access Control) را پیاده‌سازی کرده است. در این ساختار، کاربران به سه سطح اصلی تقسیم می‌شوند که هر سطح دارای مجوزها و قابلیت‌های

مشخصی است. این تفکیک تضمین می کند که هر کاربر تنها به اطلاعات و عملکردهایی دسترسی دارد که برای نقش او مجاز شمرده می شود. در ادامه، هر یک از این سطوح دسترسی به تفصیل شرح داده می شوند.

۱. **کاربر مهمان (Anonymous User):** کاربر مهمان به هر بازدیدکننده ای اطلاق می شود که بدون ورود به حساب کاربری خود در حال مرور وب سایت است. این سطح دسترسی به منظور جذب کاربر و ارائه اطلاعات عمومی طراحی شده است. سیستم، کاربران مهمان را از طریق کلید نشست (Session Key) منحصر به فرد مرورگرشان شناسایی می کند.

قابلیت ها و مجوزها:

- **مرور صفحات عمومی:** کاربران مهمان می توانند تمام صفحات عمومی سایت مانند صفحه اصلی (home)، درباره ما (about_us) و لیست محصولات (clothes_list) را مشاهده کنند.
- **مشاهده و جستجوی محصولات:** دسترسی کامل به لیست محصولات، مشاهده جزئیات هر محصول (clothe_detail)، جستجو بر اساس نام و توضیحات (search) و همچنین فیلتر و مرتب سازی محصولات برای این سطح دسترسی فراهم است.
- **استفاده از سبد خرید موقت:** کاربران مهمان می توانند محصولات را به یک سبد خرید موقت که بر اساس نشست آن ها مدیریت می شود، اضافه، ویرایش یا حذف کنند. این سبد خرید تا زمان بسته شدن مرورگر یا ورود به سیستم فعال باقی می ماند.
- **ثبت نام و ورود:** دسترسی به صفحات ثبت نام (signup) و ورود (signin) برای ایجاد حساب کاربری جدید یا ورود به حساب موجود برای این کاربران باز است.

۲. **کاربر عضو (Registered Customer):** این سطح دسترسی به کاربرانی تعلق دارد که در سایت ثبت نام کرده و با حساب کاربری خود وارد سیستم شده اند. در پایگاه داده، این کاربران دارای نقش پیش فرض customer هستند. کاربران عضو علاوه بر تمام قابلیت های کاربر مهمان، به مجموعه ای از امکانات شخصی سازی شده و مدیریتی برای حساب خود دسترسی دارند.

قابلیت ها و مجوزهای اضافی:

- **سبد خرید دائمی:** سبد خرید این کاربران به حساب کاربری آن ها متصل است و به صورت دائمی باقی می ماند. هنگام ورود، سبد خرید موقت (در صورت وجود) به صورت خودکار با سبد خرید دائمی آن ها ادغام می شود.

- **داشبورد شخصی:** کاربران عضو به یک داشبورد شخصی (dashboard) دسترسی دارند که در آن قادر به ویرایش اطلاعات پروفایل خود از طریق فرم CustomUserEditForm و تغییر رمز عبور از طریق فرم پیش فرض جنگو هستند.
- **ثبت و نهایی کردن سفارش:** این کاربران می توانند فرآیند پرداخت (checkout) را طی کرده و سفارش خود را نهایی کنند. این قابلیت تنها برای کاربران عضو فعال است.
- **مشاهده تاریخچه سفارشات:** دسترسی کامل به لیست تمام سفارش های قبلی، مشاهده جزئیات و وضعیت فعلی آن ها (مانند "در حال پردازش" یا "ارسال شده") در صفحه سفارش های من (my_orders) برای این کاربران فراهم است.

۳. **کاربر ادمین (Admin):** کاربر ادمین بالاترین سطح دسترسی در سیستم را داراست و مسئولیت مدیریت کامل وبسایت، نظارت بر عملکرد و تحلیل داده ها را بر عهده دارد. در پایگاه داده، این کاربران با نقش admin و پرچم is_staff=True مشخص می شوند. ادمین علاوه بر داشتن تمام قابلیت های یک کاربر عضو، به یک پنل مدیریتی سفارشی دسترسی دارد.

قابلیت ها و مجوزهای انحصاری:

- **دسترسی به داشبورد مدیریت:** ادمین به جای پنل مدیریت پیش فرض جنگو، به یک داشبورد سفارشی (CustomAdminSite) وارد می شود که گزارش های کلیدی و تحلیلی را نمایش می دهد.
- **مشاهده گزارش های تحلیلی:** این داشبورد شامل گزارش های جامعی مانند مجموع فروش، تعداد کل سفارشات، نمودار فروش روزانه، توزیع وضعیت سفارشات، لیست محصولات پر فروش و محصولات با موجودی کم است.
- **مدیریت کامل محصولات و دسته بندی ها:** ادمین می تواند محصولات و دسته بندی ها را ایجاد، ویرایش یا حذف کند، قیمت ها و موجودی انبار را تغییر دهد و وضعیت فعال بودن آن ها را مدیریت کند.
- **مدیریت سفارشات:** مشاهده تمام سفارش های ثبت شده در سیستم، بررسی جزئیات هر سفارش و تغییر وضعیت آن ها از طریق اقدامات سفارشی (مانند "علامت گذاری به عنوان ارسال شده") ممکن است.
- **مدیریت کاربران و نسخه ها:** ادمین به لیست تمام کاربران و نسخه های پزشکی ثبت شده دسترسی دارد و می تواند آن ها را مدیریت کند.

- مدیریت پیام‌های تماس: امکان مشاهده و مدیریت پیام‌های ارسال شده توسط کاربران از طریق فرم تماس با ما و علامت‌گذاری آن‌ها به عنوان "خوانده شده" برای ادمین فراهم است.

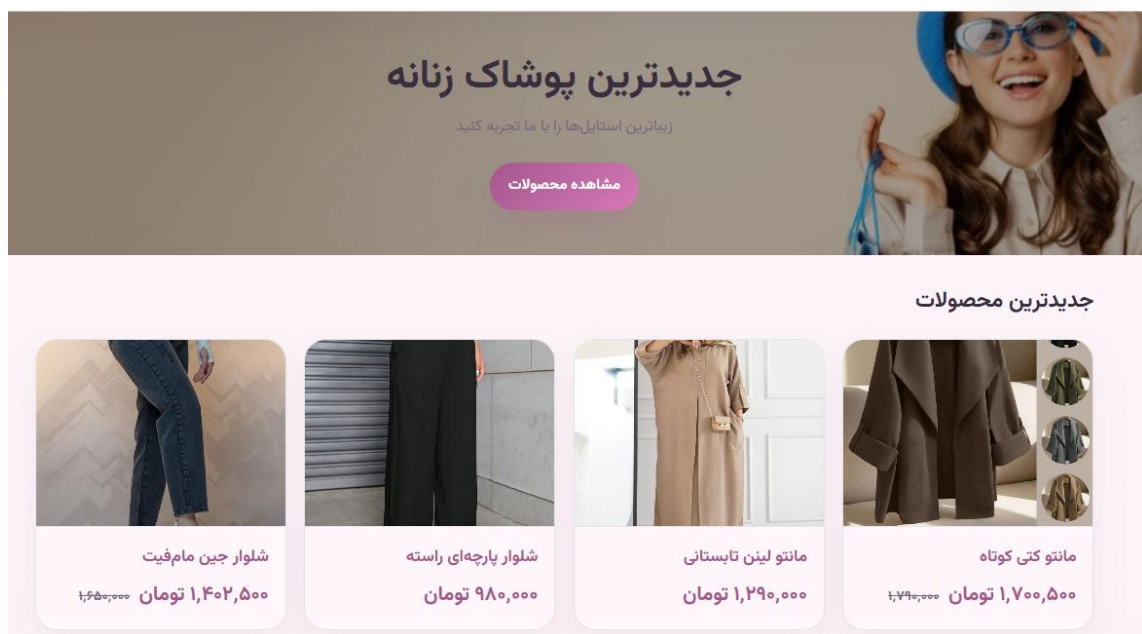
۴-۳- راهنمای صفحات پروژه

رابط کاربری وب‌سایت "women-clothing-shop" از مجموعه‌ای از صفحات HTML پویا تشکیل شده است که هر یک برای پاسخ به نیازهای یک گروه خاص از کاربران (مهمان، عضو، ادمین) طراحی شده‌اند. این صفحات با استفاده از موتور قالب جنگو (Django Template Engine) و داده‌های ارسال شده از بک‌اند، به صورت پویا رندر می‌شوند. در ادامه، راهنمای هر یک از این صفحات به تفکیک بخش‌های کاربری ارائه می‌گردد.

۴-۳-۱- صفحات در دسترس عموم (بخش مهمان)

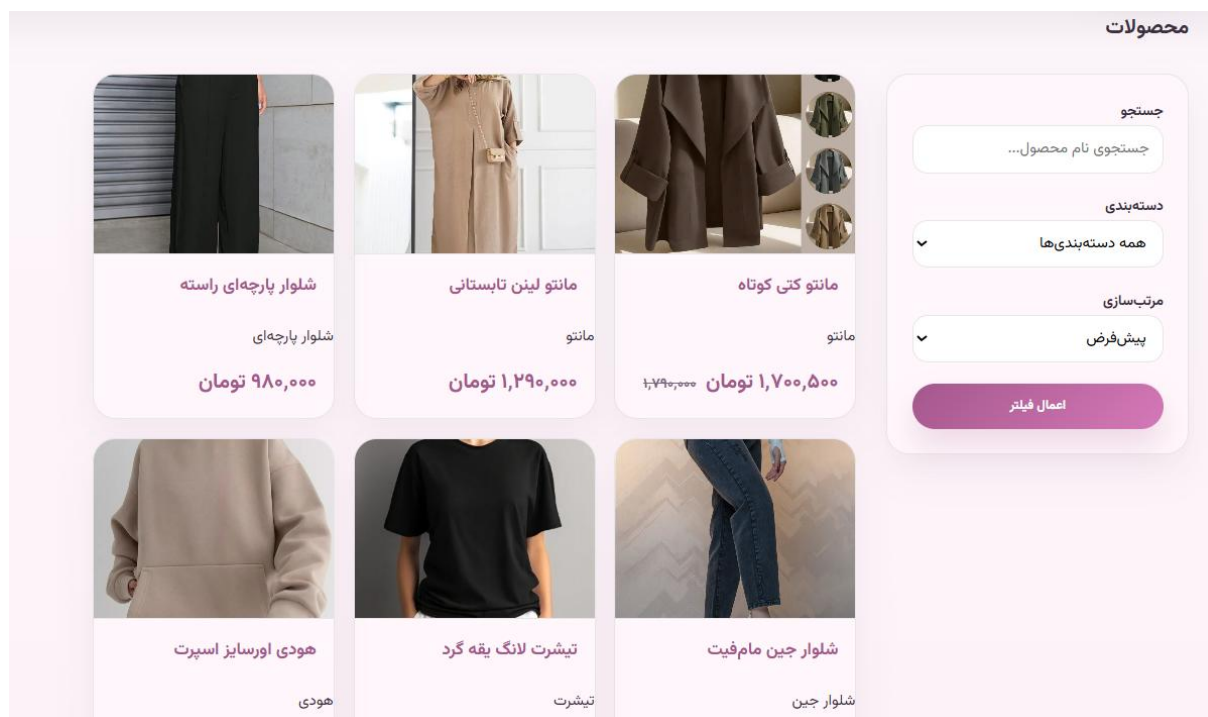
این صفحات هسته اصلی وب‌سایت را تشکیل می‌دهند و برای هر بازدیدکننده‌ای، بدون نیاز به ورود به حساب کاربری، قابل دسترس هستند.

- **صفحه اصلی (Home Page - home.html)** این صفحه نقطه ورود اصلی کاربران به وب‌سایت است. طراحی آن با هدف جذب کاربر و نمایش سریع اطلاعات کلیدی صورت گرفته است. بر اساس منطق پیاده‌سازی شده در `core/views.py`، این صفحه شامل لیستی از جدیدترین محصولات اضافه شده (`newest_clothes`) و محصولاتی که دارای بیشترین تخفیف هستند (`highest_discounts`) می‌باشد تا پیشنهادات ویژه را در همان نگاه اول به کاربر نمایش دهد.



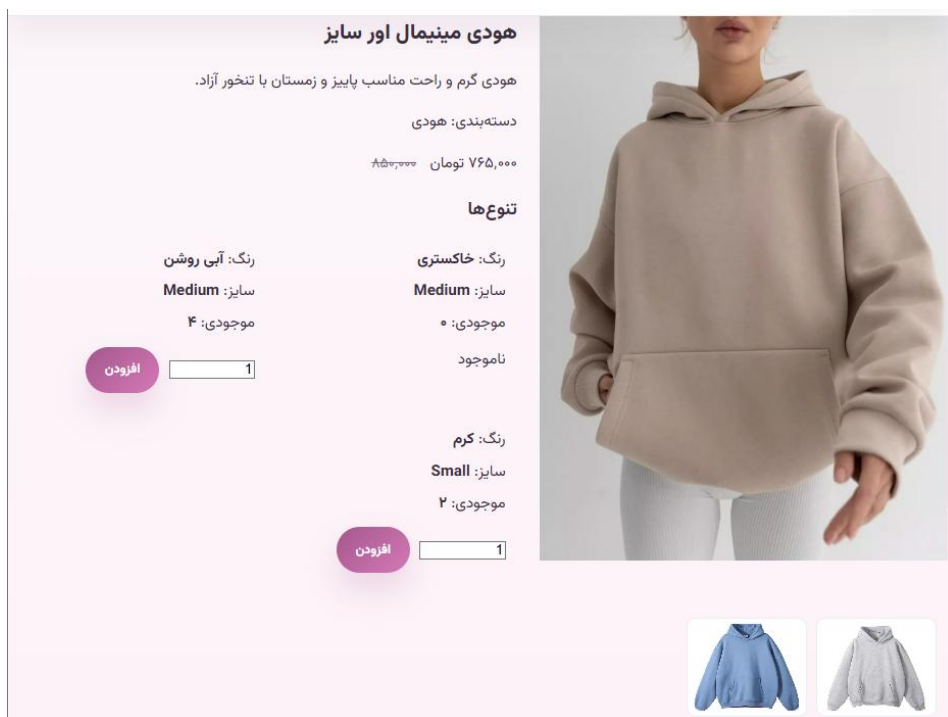
شکل ۲-۴ نمای صفحه‌ی خانه

- **صفحه فروشگاه (Shop Page – clothes_list.html)** این صفحه، ویتترین اصلی محصولات فروشگاه است. نکته قابل توجه این است که قالب clothes_list.html یک قالب چندمنظوره بوده و توسط نماهای مختلفی برای نمایش لیست محصولات استفاده می‌شود: نمایش تمام محصولات، نمایش محصولات یک دسته‌بندی خاص، و نمایش نتایج جستجو. این صفحه به قابلیت‌های پیشرفته‌ای برای بهبود تجربه کاربری مجهز است:
 - **صفحه‌بندی (Pagination):** محصولات در صفحات ۱ تا ۱۲ نمایش داده می‌شوند تا از بارگذاری کند صفحه جلوگیری شود.
 - **مرتب‌سازی (Sorting):** کاربر می‌تواند محصولات را بر اساس قیمت (ارزان‌ترین/گران‌ترین) و جدیدترین‌ها مرتب کند.



شکل ۳-۴ نمای صفحه‌ی فروشگاه

- صفحه جزئیات محصول (Product Detail Page – `clothe_detail.html`) هر محصول دارای یک صفحه اختصاصی برای نمایش تمام جزئیات آن است. این صفحه اطلاعات کاملی مانند نام، توضیحات، قیمت، سایز و رنگ (واریانت‌ها) و تصاویر محصول را نمایش می‌دهد. همچنین، این صفحه بررسی می‌کند که آیا محصول در سبد خرید کاربر قرار دارد یا خیر و اطلاعات مربوط به آن را نمایش می‌دهد تا فرآیند تصمیم‌گیری برای خرید را آسان‌تر کند.



شکل ۴-۴ نمای صفحه‌ی جزئیات محصول

- صفحه سبد خرید (Cart Page – cart_detail.html) کاربران (اعم از مهمان و عضو) می‌توانند محتویات سبد خرید خود را در این صفحه مشاهده کنند. این صفحه لیست محصولات اضافه شده، تعداد هر کدام، قیمت واحد و قیمت کل سفارش را نمایش می‌دهد. برای کاربران مهمان، اطلاعات از سبد خرید موقت مبتنی بر نشست (Session) خوانده می‌شود.



شکل ۴-۵ نمای صفحه‌ی سبد خرید

- صفحات ورود و ثبت نام (login.html و register.html) این دو صفحه برای مدیریت فرآیند احراز هویت طراحی شده‌اند. صفحه register.html فرم CustomUserCreationForm را برای ایجاد حساب کاربری جدید نمایش می‌دهد، در حالی که صفحه login.html فرم AuthenticationForm را برای ورود کاربران موجود ارائه می‌کند.
- صفحه درباره ما (about_us.html) این صفحه برای ارائه اطلاعات درباره فروشگاه طراحی شده است.

ثبت نام
ساخت حساب جدید

نام کاربری:

الزامی. 150 کاراکتر یا کمتر. فقط شامل حروف، اعداد، و علامات @/./-/_/.

آدرس ایمیل:

گذرواژه:

- گذرواژه شما نمی‌تواند شبیه سایر اطلاعات شخصی شما باشد.
- رمز عبور شما می‌بایست حداقل از 8 حرف تشکیل شده باشد.
- گذرواژه شما نمی‌تواند یک گذرواژه معمول باشد.
- گذرواژه شما نمی‌تواند کلاً عدد باشد.

تأیید گذرواژه:

برای تائید، رمز عبور قبلی را وارد کنید.

ثبت نام

شکل ۶-۴ نمای صفحه‌ی ثبت نام

۴-۳-۲- صفحات اختصاصی کاربر عضو

این صفحات تنها برای کاربرانی که با حساب کاربری خود وارد سیستم شده‌اند (@login_required) قابل دسترس هستند و امکانات شخصی‌سازی شده و مدیریتی را فراهم می‌کنند.

• داشبورد کاربری (User Dashboard – account_dashboard.html) این صفحه، مرکز کنترل

حساب کاربری است. در این صفحه، اطلاعات کاربری نشان داده شده است و یک sidebar وجود

دارد که شامل لینک به سه صفحه مجزا برای مدیریت جنبه‌های مختلف حساب کاربری است:

۱. ویرایش اطلاعات: انتقال به صفحه edit_account برای ویرایش اطلاعات شخصی مانند نام

و شماره تلفن بوسیله فرم CustomUserEditForm.

۲. تغییر رمز عبور: انتقال به صفحه change_password برای تغییر امن رمز عبور بوسیله

PasswordChangeForm.

The screenshot displays a user dashboard with a light pink background. At the top right, there is a section titled 'داشبورد کاربری' (User Dashboard) with a subtitle 'سلام USER، خوش آمدید.' (Hello USER, welcome). Below this, there are three input fields for user information: 'ایمیل' (Email) with the value 'user@gmail.com', 'نام کاربری' (Username) with the value 'user', and 'شماره تلفن' (Phone Number) with the value '09123456789'. To the right of these fields, there is a list of navigation links: 'داشبورد' (Dashboard), 'سفارش‌های من' (My Orders), 'ویرایش اطلاعات' (Edit Information), and 'تغییر رمز عبور' (Change Password).

شکل ۷-۴ نمای صفحه‌ی داشبورد

- **صفحه پرداخت (Checkout Page - checkout.html)** این صفحه گام نهایی در فرآیند خرید است و تنها در صورتی که سبد خرید کاربر خالی نباشد، نمایش داده می‌شود. در این صفحه، کاربر اطلاعات حمل و نقل را وارد می‌کند.

The screenshot shows a checkout page with a light pink background. At the top, there's a header with 'تسویه حساب' (Settle Account) and 'اطلاعات ارسال' (Shipping Information). Below this, there's a form for shipping address with a label 'آدرس ارسال:' and a text input field with a placeholder 'آدرس پستی خود را وارد کنید...'. Below the form, there are two buttons: 'بازگشت به سبد خرید' (Return to Shopping Cart) and 'ثبت سفارش' (Place Order). Below the buttons, there's a section titled 'خلاصه سفارش' (Order Summary) which lists two items: 'هودی مینیمال اور سائیز - کرم - Small' and 'هودی مینیمال اور سائیز - آبی روشن - Medium'. Each item has a price of ۷۶۵,۰۰۰ تومان. At the bottom, there's a total amount: 'جمع کل: ۱,۵۳۰,۰۰۰ تومان'.

شکل ۸-۴ نمای صفحه‌ی پرداخت

- **صفحه تاریخچه سفارشات (Orders Page – my_orders.html)** کاربران عضو می‌توانند در این صفحه لیست کاملی از تمام سفارش‌های قبلی خود را مشاهده کنند. اطلاعاتی مانند شماره سفارش، تاریخ، مبلغ کل و وضعیت فعلی هر سفارش (مانند "ارسال شده" یا "تحویل داده شده") در این

صفحه نمایش داده می‌شود. برای بهینه‌سازی، این صفحه از prefetch_related برای بارگذاری بهینه اطلاعات استفاده می‌کند.

سفارش‌های من

لیست سفارش‌های ثبت‌شده شما

Processing

سفارش شماره ۱۰

29 مه 2026، ساعت 7:32

محصولات سفارش

مانتو کتی کوتاه

سبز یشمی | Medium



x1

۱,۷۰۰,۵۰۰ تومان

شلوار جین مام‌فیت

آبی روشن | Medium



x2

۲,۸۰۵,۰۰۰ تومان

تعداد آیتم‌ها:

۲ مبلغ کل:

۴,۵۰۵,۵۰۰ تومان

شکل ۹-۴ نمای صفحه‌ی تاریخچه سفارشات

۴-۳-۳- صفحات اختصاصی ادمین

این بخش صرفاً به پنل مدیریت اختصاص دارد که در مسیر `/manage/` قابل دسترسی است. مسیر پیش فرض جنگو یعنی `/admin/` هنوز وجود دارد، اما رابط مدیریتی اختصاصی این پروژه از طریق `/manage/` پیاده‌سازی شده است. یکی از ویژگی‌های برجسته‌ی این پروژه پنل مدیریت اختصاصی است که در مسیر

/manage/ قرار دارد. ادمین‌ها می‌توانند با ورود از مسیر سایت /login/ و یا صفحه مخصوص مدیریت /manage/login/ به داشبورد مدیریتی وارد شوند.

• داشبورد مدیریت (Admin Dashboard – admin_dashboard.html) این صفحه، نقطه

ورود و مرکز فرماندهی ادمین است و جایگزین صفحه پیش‌فرض ادمین جنگو شده است:

○ دسترسی به صفحات مدیریت کاربران، مشاهده لیست تمام سفارشات و ویرایش وضعیت

سفارش‌ها، گزارش‌گیری حرفه‌ای و افزودن محصول (برای ویرایش و حذف محصول، باید

وارد صفحه خود محصول مربوطه شویم).



شکل ۱۰-۴ نمایی از داشبورد ادمین

مدیریت تمام سفارش‌ها						
شناسه سفارش	کاربر	تاریخ	مجموع (تومان)	وضعیت	عملیات	
10	user	07:32 2026-05-29	۴,۵۰۵,۵۰۰	Shipped	بگوررسانی	
9	user	05:40 2026-05-29	۳,۸۲۵,۰۰۰	Processing	بگوررسانی	
8	user	02:22 2026-05-29	۷۶۵,۰۰۰	Cancelled	بگوررسانی	
7	user	17:49 2026-05-28	۲,۲۹۵,۰۰۰	Shipped	بگوررسانی	
6	user	15:09 2026-05-28	۴,۸۳۰,۰۰۰	Processing	بگوررسانی	

شکل ۱۱-۴: نمایی از صفحه ویرایش ادمین

۴-۴- نتیجه گیری

پروژه حاضر با هدف طراحی و پیاده‌سازی یک سامانه جامع فروشگاه آنلاین پوشاک زنانه تحت عنوان "women-clothing-shop" با موفقیت به اتمام رسید. این سیستم که با استفاده از زبان برنامه‌نویسی پایتون و فریم‌ورک قدرتمند جنگو توسعه یافته است، توانست به طور مؤثری به چالش‌های اصلی مطرح شده در فرآیند تحلیل، یعنی رفع محدودیت‌های داروخانه‌های سنتی و پاسخ به نیازهای کاربران در دنیای دیجیتال، پاسخ دهد. نتیجه نهایی، یک وب اپلیکیشن کاملاً کاربردی و یکپارچه است که فرآیندهای خرید و مدیریت را به شکل چشمگیری بهینه کرده است.

- در این پروژه ساختار داده‌ای شفاف و منظم با ORM جنگو طراحی شده است. موجودیت‌های کلیدی شامل:

- CustomUser: با سطوح دسترسی متفاوت و فیلدهای کاربری سفارشی
- Clothe: برای محصولات اصلی
- ClotheVariant: برای تنوع‌های رنگ و سایز هر محصول
- ClotheImage: برای گالری تصاویر محصولات
- Cart: به صورت مدیریت شده در سشن
- Order و OrderItem: برای ثبت کامل سفارش‌ها

- فرآیندهای مهم کسب‌وکار نیز پیاده‌سازی شده‌اند:

- سیستم احراز هویت با کاربری سفارشی و مدیریت حساب کاربران
- سبد خرید هوشمند بر پایه‌ی سشن
- جریان سفارش‌دهی کامل با محاسبه‌ی صحیح قیمت، تعداد و مجموع

در نهایت، پروژه "women-clothing-shop" با رعایت اصول مهندسی نرم‌افزار، از جمله

کدنویسی تمیز، مستندسازی دقیق و پیاده‌سازی راهکارهایی برای تضمین یکپارچگی داده‌ها (مانند تراکنش‌های اتمی)، به یک محصول نرم‌افزاری قابل اعتماد و قابل نگهداری تبدیل شده است. این سیستم به خوبی نشان می‌دهد که چگونه می‌توان با استفاده از ابزارهای مناسب، یک راهکار دیجیتال کامل برای یک حوزه تخصصی و حساس مانند داروسازی ارائه داد.

۴-۵- پیشنهادات

با وجود موفقیت پروژه در تحقق اهداف اولیه، همواره زمینه‌هایی برای بهبود و توسعه قابلیت‌های جدید وجود دارد. ارائه پیشنهادات زیر می‌تواند به ارتقای کیفیت فنی و افزایش ارزش کاربردی سیستم در آینده کمک کند:

الف) توسعه و بهبودهای فنی

- **پیاده‌سازی تست‌های خودکار (Automated Tests):** فایل‌های tests.py پروژه در حال حاضر خالی هستند. پیشنهاد می‌شود که مجموعه‌ای جامع از تست‌های واحد (Unit Tests) و تست‌های یکپارچه‌سازی (Integration Tests) برای اعتبارسنجی عملکرد مدل‌ها، نماها و فرم‌ها نوشته شود. این کار پایداری سیستم را در برابر تغییرات آتی تضمین کرده و فرآیند خطایابی را به شدت تسریع می‌کند.
- **بهینه‌سازی برای محیط پروداکشن:** در حال حاضر، پروژه با تنظیمات توسعه (مانند DEBUG=True و پایگاه داده SQLite) پیکربندی شده است. پیشنهاد می‌شود یک فایل تنظیمات مجزا برای محیط پروداکشن ایجاد شود که در آن از پایگاه داده قدرتمندتری مانند PostgreSQL استفاده شده، کلیدهای امنیتی به صورت متغیرهای محیطی مدیریت شوند و تنظیمات مربوط به عملکرد و امنیت برای استقرار نهایی بهینه گردند.

ب) توسعه قابلیت‌های جدید

- **یکپارچه‌سازی با درگاه پرداخت آنلاین:** فرآیند ثبت سفارش، در حال حاضر بدون پرداخت و صرفاً با اضافه کردن آدرس پستی انجام می‌شود. مهم‌ترین قابلیت جدیدی که می‌توان اضافه کرد، اتصال سیستم به یک یا چند درگاه پرداخت آنلاین است تا کاربران بتوانند هزینه سفارش خود را به صورت الکترونیکی پرداخت کرده و سفارش آن‌ها به صورت خودکار تأیید شود.
- **سیستم اطلاع‌رسانی به کاربر:** پیشنهاد می‌شود یک سیستم اطلاع‌رسانی از طریق ایمیل یا پیامک به پروژه اضافه شود. این سیستم می‌تواند کاربران را از رویدادهای مهمی مانند ثبت‌نام موفق، ثبت سفارش جدید و تغییر وضعیت سفارش (مثلاً از "در حال پردازش" به "ارسال شده") مطلع سازد.
- **سیستم نظردهی:** اضافه کردن ویژگی نظرات و قابلیت نظر دادن برای هر محصول.
- **قابلیت‌های پیشرفته در داشبورد ادمین:** داشبورد مدیریت فعلی بسیار کارآمد است، اما می‌توان با افزودن قابلیت‌هایی مانند خروجی گرفتن از گزارش‌ها در فرمت‌های Excel یا PDF و افزودن فیلترهای پیشرفته‌تر برای گزارش‌گیری (مانند فیلتر بر اساس کاربر یا محصول خاص)، آن را قدرتمندتر کرد.

٤-٦ - منابع و مراجع

١. Stein, Clara. Django & Cookbook: ٧٠+ Problem Solving Techniques, Sample Programs, and Troubleshoots Across Python Programs and Web Apps. GitforGits, ١٥ March ٢٠٢٤. ٢١٤ pages.
٢. Vincent, William S. Django for Beginners, ٥th Edition: Build Modern Web Applications with Python. Still River Press, ١٠ July ٢٠٢٤. ٣٥٧ pages.
٣. Duckett, Jon. Web Design with HTML, CSS, JavaScript and jQuery Set. Wiley, ١st edition, ٨ July ٢٠١٤. ١١٥٢ pages.
٤. "Django makes it easier to build better web apps more quickly and with less code." Available: <https://www.djangoproject.com>
٥. django project, "Working with forms". Available: <https://docs.djangoproject.com/en/٥.١/topics/forms/>.
٦. django project, "Models". Available: <https://docs.djangoproject.com/en/٥.١/topics/db/models/>.
٧. django project, "Templates". Available: <https://docs.djangoproject.com/en/٥.١/topics/templates/>.
٨. django project, "How to create custom template tags and filters". Available: <https://docs.djangoproject.com/en/٥.١/howto/custom-template-tags/>.
٩. django project, "Settings". Available: <https://docs.djangoproject.com/en/٥.١/ref/settings/>.
١٠. django project, "django.urls utility functions". Available: <https://docs.djangoproject.com/en/٥.١/ref/urlresolvers/>.
١١. django project, "Source code for django.urls.converters". Available: https://docs.djangoproject.com/en/٣.٢/_modules/django/urls/converters/.
١٢. django project, "How to create custom django-admin commands". Available: <https://docs.djangoproject.com/en/٥.١/howto/custom-management-commands/>.
١٣. wikipedia, "SQLite". Available: <https://en.wikipedia.org/wiki/SQLite>.
١٤. django project, "How to manage static files (e.g. images, JavaScript, CSS)". Available: <https://docs.djangoproject.com/en/٥.١/howto/static-files/>.
١٥. django project, "Managing files". Available: <https://docs.djangoproject.com/en/٥.١/topics/files/>